

Available online at www.sciencedirect.com

ScienceDirect

journal homepage: www.elsevier.com/locate/coseComputers
&
Security

A survey of security solutions for distributed publish/subscribe systems



CrossMark

Anton V. Uzunov *

Defence Science and Technology Group (DST Group), West Ave., Edinburgh, SA 5111, Australia

ARTICLE INFO

Article history:

Received 4 July 2015

Received in revised form 1 February 2016

Accepted 27 April 2016

Available online 4 May 2016

Keywords:

Software security

Publish/subscribe

Distributed event-based systems

Software architecture

Security architecture

Survey

ABSTRACT

Over the last two decades, the need for loosely-coupled, asynchronous communications and distributed component interaction has made the *publish/subscribe* pattern increasingly popular in a variety of contexts. Despite their popularity, systems realizing this pattern are inherently susceptible to a wide variety of security threats, both of a general and specific nature. An increasing awareness of the importance of countering these threats has instigated a number of relevant proposals in the literature in the form of various types of *security solutions*. Having a broad knowledge of such security solutions is a prerequisite not only to proposing new solutions, but also to further advancing, re-using and adapting existing solutions to future *publish/subscribe* architectures. This paper seeks to contribute to the satisfaction of the latter prerequisite by presenting a comprehensive state-of-the-art survey of security solutions for distributed *publish/subscribe* systems, focusing on solutions taking the form of concrete security architectures. The solutions are reviewed “horizontally” from an architectural viewpoint, allowing their constituent parts to be seen in their original contexts. These “horizontal” reviews are augmented with brief analyses of constituent security patterns and pertinent threats addressed, thus providing a contextualized “vertical” dimension to each solution individually. In this way the paper provides a distinct perspective on a rapidly growing area of research, while complementing and consolidating the existing body of work concerned with surveying *publish/subscribe* security in general.

Crown Copyright © 2016 Published by Elsevier Ltd. All rights reserved.

1. Introduction

Over the last two decades, the need for loosely-coupled, asynchronous communications and distributed component interaction has made the *publish/subscribe* pattern increasingly popular in a variety of contexts (Mühl et al., 2006; Opyrchal and Prakash, 2003; Tarkoma, 2013), ranging from complex event processing and status dissemination in cyber-physical infrastructures and health-care applications (Bakken et al., 2011; Cristea et al., 2011; Paschke et al., 2012; Singh and Bacon, 2009) to alert correlation in autonomous intrusion response systems

(Docking et al., 2015; García et al., 2005). Being distributed systems in their own right, realizations of the *publish/subscribe* pattern are inherently susceptible to all the security threats faced by general distributed systems (network- and host-based), as well as a number of more specific threats (Aniello et al., 2014; Wang et al., 2002), which, as Savinov and Alencar (2014) and Fotiou et al. (2010a) point out, stem predominantly from the fact that attackers can leverage loosely-coupled *publish/subscribe* components as “entry points” for launching attacks. The inherent vulnerabilities in *publish/subscribe* systems become particularly noxious when requirements exist in a given system’s context for sensitive

* Tel.: +618 7389 7252.

E-mail address: anton.uzunov@dsto.defence.gov.au.<http://dx.doi.org/10.1016/j.cose.2016.04.008>

0167-4048/Crown Copyright © 2016 Published by Elsevier Ltd. All rights reserved.

information to pass through untrusted, third-party networks or across disparate administrative domains (Bacon et al., 2009) – as is the case for all the contexts adduced as examples above; or when a given publish/subscribe system supports security and/or safety critical infrastructures, or is security and/or safety critical itself – as is also the case for most of the aforementioned systems.

An increasing awareness of the importance of security has instigated a number of relevant proposals in the literature in the form of various types of security solutions, such as the security architectures of concrete publish/subscribe systems, as well as generic, system-independent frameworks and isolated solutions, addressing one or more pertinent security threats. This trend has been so strong over the last nine years in particular, that a generally valid statement made by Corsaro et al. in 2006, namely, “security issues represent one major problem in pub/sub systems, only marginally addressed at present by both researchers and industry” (Corsaro et al., 2006) – and repeated in different forms in many topical security-related publications – is no longer true, with a significant body of research now attesting to the contrary. Having a broad knowledge of this research is a prerequisite not only to proposing new security solutions, but also to further advancing, re-using and adapting existing security solutions for future publish/subscribe architectures.

Naturally, this prerequisite is best satisfied by a comprehensive survey of the given area. Yet in contrast to the relative profusion of surveys and related work considering the general functional and other non-functional features of publish/subscribe systems – e.g. Bellavista et al. (2014); Esposito et al. (2013); Eugster (2007); Eugster et al. (2003); Liu and Plale (2003); Martins and Duarte (2010); Tarkoma and Raatikainen (2006) – the current literature presents only a single survey on publish/subscribe security: that of Esposito and Ciampi (2015) from late 2014. Although Esposito and Ciampi’s survey is comprehensive and covers much ground, its prevalent focus is on abstract, system-independent solutions – as opposed to the security architectures of concrete systems – which means that a number of solutions are not covered. Moreover, the solutions which are included are presented in a “vertical” fashion, with each solution divided across several security concerns – a presentation which certainly has advantages from the point of view of overviewing the area, but one which does not allow solutions to be seen in their original contexts, making it difficult to gain insight into how the solution parts fit together and hence how similar solutions should be realized in real-life systems (we explain the distinctions between Esposito and Ciampi (2015) and this paper in more detail later on).

The present paper aims to rectify the aforementioned omissions and complement the recent work of Esposito and Ciampi (2015) by providing a comprehensive, state-of-the-art survey of security solutions (understood as individual or groups of concrete countermeasures addressing one or more pertinent security threats – in essence security pattern instances (Fernandez, 2013; Uzunov et al., 2012a) or groups of such instances) for distributed publish/subscribe systems, focusing on the security architectures of concrete systems. The solutions are reviewed “horizontally” and from an architectural viewpoint, allowing their constituent parts to be seen in their original contexts. These “horizontal” reviews are augmented with brief

analyses of constituent security patterns and pertinent threats addressed, thus providing a contextualized “vertical” dimension to each solution individually. In this way the survey aims to maximally contribute to the satisfaction of the prerequisite of creating a sound and holistic knowledge-base to support, above all, the design of future publish/subscribe security architectures as referred to above – while providing a distinct viewpoint on a rapidly growing area of research.

The rest of the paper has been structured as follows. Section 2 discusses related work in the form of surveys and taxonomies. Section 3 provides a brief overview and presents a conceptual reference model of publish/subscribe systems; describes relevant security threats via an abstract threat model; and defines our classification of security solutions. Section 4 presents a series of reviews of concrete security architectures in some detail; and Section 5 reviews several frameworks and isolated solutions that are not covered in Esposito and Ciampi (2015), in brief summary form. In all cases the aim is to foster some degree of appreciation and understanding of the specific features of the security solutions under review, as opposed to merely noting their existence in passing. We are firmly convinced more in-depth reviews (especially of security architectures, following our focus) are necessary to dispel the notion alluded to already that security for publish/subscribe systems has received little attention – a notion which continues to prevail in many publications appearing even close to the time of this writing.

As mentioned earlier, all security solutions can be seen as collections of security pattern instances, and throughout Sections 4 and 5 we make the latter relation explicit by indicating the main patterns (of those available at the time of this writing) constituting each solution, and which threats – from the threat model introduced in Section 2 – these patterns seek to address. Besides providing a “vertical” dimension to the reviews as mentioned already, these brief analyses serve the dual purpose of summaries and evaluations with respect to the threats the security solutions address (cf. (Dikanski et al., 2012; Fernandez et al., 2016)). Knowing which security patterns comprise a solution is also important when applying architectural analysis techniques such as those of Halkidis et al. (2008) or Heyman et al. (2008), not only for the analysis of particular solutions, but of whole publish/subscribe architectures implementing those solutions. The set of reviewed solutions is subsequently compared based on the threats they address in Section 6, where we also identify gaps and advance ideas for improving publish/subscribe security solutions in general. Finally, in Section 7 we conclude and discuss future research directions stemming from this survey.

2. Related work

As intimated in the preceding discussions, there are a number of surveys relating to the functional aspects of publish/subscribe systems or to their taxonomization. Some prominent examples include:

- Eugster et al. (2003), which places publish/subscribe in the context of other distributed system communication/

interaction paradigms, such as remote method invocation and asynchronous messaging;

- Tarkoma and Raatikainen (2006), appearing later in extended form in Tarkoma (2013), which provides an overview (in variable depth) of the prominent features of numerous research and industrial publish/subscribe systems;
- Baldoni et al.'s (2009) comprehensive, high-level survey and taxonomy of publish/subscribe routing algorithms;
- The taxonomies of Liu and Plale (2003), Meier and Cahill (2005), Mahambre et al. (2007), Blanco and Alencar (2010), Mayer et al. (2012), Hummer et al. (2012), which collectively capture a very wide spectrum of publish/subscribe system characteristics;
- Background literature reviews in various PhD and Master-level dissertations (e.g. Virgilio, 2003; Meier, 2003; Fidler, 2006; Jaeger 2007; Blanco, 2010; Fischer, 2014) as well as relevant chapters in books on event-based systems and related topics (e.g. Mühl et al., 2006; Tarkoma, 2009).

While these publications form an extremely comprehensive outline of publish/subscribe, they uniformly consider security as a peripheral attribute. The background sections and literature reviews in PhD and Master-level dissertations focused in some fashion and to some degree on security aspects of publish/subscribe systems, such as Altaweel (2009), Dionysiou (2006), Nabeel (2012), Pesonen (2008), Tariq (2013), and Yoon (2013), clearly consider security as a central attribute, but are far too brief to be considered substantial in their own right.

Other relevant surveys that focus on Quality of Service (QoS) guarantees provided by different publish/subscribe systems are relatively consistent in including discussions of security at least in a broad sense; however, whenever security is considered, this is done without reviewing concrete security architectures or solutions from the literature in any significant detail. Some examples of surveys and overview papers in this category include:

- Corsaro et al. (2006) which, besides reviewing JMS and DDS (two industry publish/subscribe systems/architectures), mentions some of the work of Dionysiou (2006) and Fiege et al. (2004) in a general context discussing the importance of security;
- Behnel et al. (2006), which briefly discusses issues affecting confidentiality (encryption) as well as authentication and integrity in a general setting, with passing references to several abstract and concrete solutions, such as those found in HERMES;
- Bellavista et al. (2014), which discusses security together with several other important QoS properties and provides comprehensive, in-depth reviews of a small selection of systems, but without reviewing their security features.

The publish/subscribe literature also presents relevant surveys focusing on reliability and fault-tolerance attributes, such as Esposito et al. (2013), Hummer et al. (2012), and Mayer et al. (2012). Reliability does overlap with security with respect to availability, however, the aforementioned works and, for the most part, the solutions they survey, are based on the fundamentally distinct viewpoint that faults – hence also faults arising from intrusions – are caused accidentally and not intention-

ally (Esposito and Ciampi, 2015). It is not surprising, therefore, that these surveys do not consider attributes strictly relating to security, for the exception of Mayer et al. (2012), who include message tampering, message misuse, information leaks and selfish behaviours as part of their fault model (and, indeed, reproach an earlier version of Esposito et al. (2013) for omitting such categories). The result of Mayer et al.'s characterization of a number of existing publish/subscribe systems with respect to their fault model suggests that almost none of the systems (over 20) account for these essentially security-related faults – which is true when looked at from a purely fault-tolerance perspective. However, some of the systems under consideration do indeed account for the aforementioned faults (e.g. cf. Section 4.1.3 of this paper), but present the relevant solutions in security-related publications, which were probably omitted during the survey process. In any case, it would not be correct to interpret any reliability-/fault-tolerance-related surveys as substitutes for a survey focusing on security solutions explicitly, even though they can be seen as supplementary with respect to availability (at least when considering Byzantine fault tolerance – cf. Esposito and Ciampi, 2015).

This last point in particular is affirmed in the taxonomy of Wun et al. (2007), which represents the sole (thus far) publication devoted singly to a security attribute of some kind specifically in the context of publish/subscribe systems. However, the latter publication only considers threats, not solutions. In a somewhat similar vein, one can mention the seminal paper by Wang et al. (2002), which provides a high-level overview of security threats specific to publish/subscribe systems and discusses several possible solutions in the abstract. While it could be seen as a kind of “precursor” to this survey, Wang et al.'s work was initiated at a time (ca. 2002) when the statement of Corsaro and colleagues cited in the Introduction (Section 1) – to the effect that there is little research in publish/subscribe security – was unequivocally true (cf. Opyrchal and Prakash, 2001), hence the distinct lack of focus on concrete architectures or implemented and tested security solutions is (or at least can be) justified. Needless to say, neither Wang et al.'s overview nor the taxonomy of Wun et al. referred to above – nor the related work sections in various relevant publications cited in this paper, which can be considered brief, partial, naturally one-sided overviews in their own right – are surveys *per se*.

All the aforementioned references suggest that, while publish/subscribe systems have received significant attention in the survey literature (and related publications), in the period prior to November 2014 there has been little work that provides a focused overview of security solutions for this type of systems.

The first comprehensive and currently sole survey focusing on security in (general) publish/subscribe systems can be found in Esposito and Ciampi (2015), and is, for obvious reasons, the most closely related to this paper of all previous work. Esposito and Ciampi's survey is distinct from the present one in two major respects: focus and selection of solutions.¹

¹ It should be pointed out that the present survey was begun in May 2014, and was reaching an advanced stage of completion when the survey of Esposito and Ciampi was published (online November 2014).

As mentioned in the Introduction, with respect to its overview of academic research, Esposito and Ciampi's survey focuses predominantly on abstract, system-independent security solutions, and presents them "vertically", divided across the categories of a taxonomy of security concerns, with the ultimate aim being to point out gaps and improvements with respect to the reviewed solution set. While this approach provides a clear overview of how each given concern is addressed in each chosen security solution, it does not provide a holistic view of the solutions, and hence it is more difficult to envisage how the constituent parts fit, or should fit, together. The present survey envisages a security solution (Section 3.3) as a more encompassing notion and focuses on one category of such solutions – the security architectures of concrete systems – which are reviewed "horizontally" and from an architectural viewpoint, thus providing a holistic view. The analysis of each solution via security patterns and threats addressed also provides an additional "vertical" dimension – but in the context of the individual solutions, thus making it easier to see how the constituent parts fit, or should fit, together when constructing similar, or new, publish/subscribe security architectures, respectively.

The focus on security architectures in the present survey also implies that the vast majority of solutions chosen for detailed review do not overlap with those reviewed by Esposito and Ciampi (in total 10 out of 13 major security architectures, in addition to 7 out of 8 "supplemental" abstract, system-independent solutions, are distinct). It goes without saying that those which do overlap are reviewed quite differently. Since our focus is also on *distributed* publish/subscribe systems, of the solutions present in industrial systems we only consider DDS (Data Distribution Service) Security (OMG, 2014a) in relation to RTI's Connex DDS, an OMG (Object Management Group) standard that was not available in its final form when Esposito and Ciampi were writing their survey. Besides DDS Security, it would have been superfluous to repeat Esposito and Ciampi's review of WSN (Web Service Notification) (OASIS, 2006).

The above discussion underscores the fact that the present survey and the survey of Esposito and Ciampi are in fact *complementary*, and provide different views of the same subject area that are valuable for different purposes, with different approaches and with reviews of a surprisingly non-overlapping set of security solutions.

3. Background

In this section we provide an overview of publish/subscribe systems, explaining how their architectures are organized (Section 3.1), as well as what threats they face (Section 3.2) and the categories of solutions that have been created to address those threats (Section 3.3).

3.1. Overview of publish/subscribe systems

3.1.1. Basic definitions

Publish/subscribe systems employ, or can be entirely described as, instances of the *Publish/Subscribe architectural pattern* (Buschmann et al., 2007), which encapsulates an approach for asynchronous, anonymous, many-to-many, loosely-coupled

communication and component interaction based on an event-driven paradigm (Eugster et al., 2003; Mühl et al., 2006; Tarkoma, 2013). Event-driven (or, equivalently, event-based) in this context implies that all "object interactions are modelled as events, which are transmitted in the form of notifications" (Sutton et al., 2001) to interested components – where events can be equated with incidents or "data-rich occurrences, encapsulating a particular semantic" (Singh and Bacon, 2009). Although they are distinct, when speaking of events the terms "notifications" and "messages" can largely be used synonymously, since events are transported through the communication media as messages, which in turn essentially notify each subscriber (and/or intermediate node) of the encapsulated event; and also since the latter terms are used in the literature by different authors to mean or to imply equivalent concepts.

3.1.2. Conceptual reference model of publish/subscribe systems

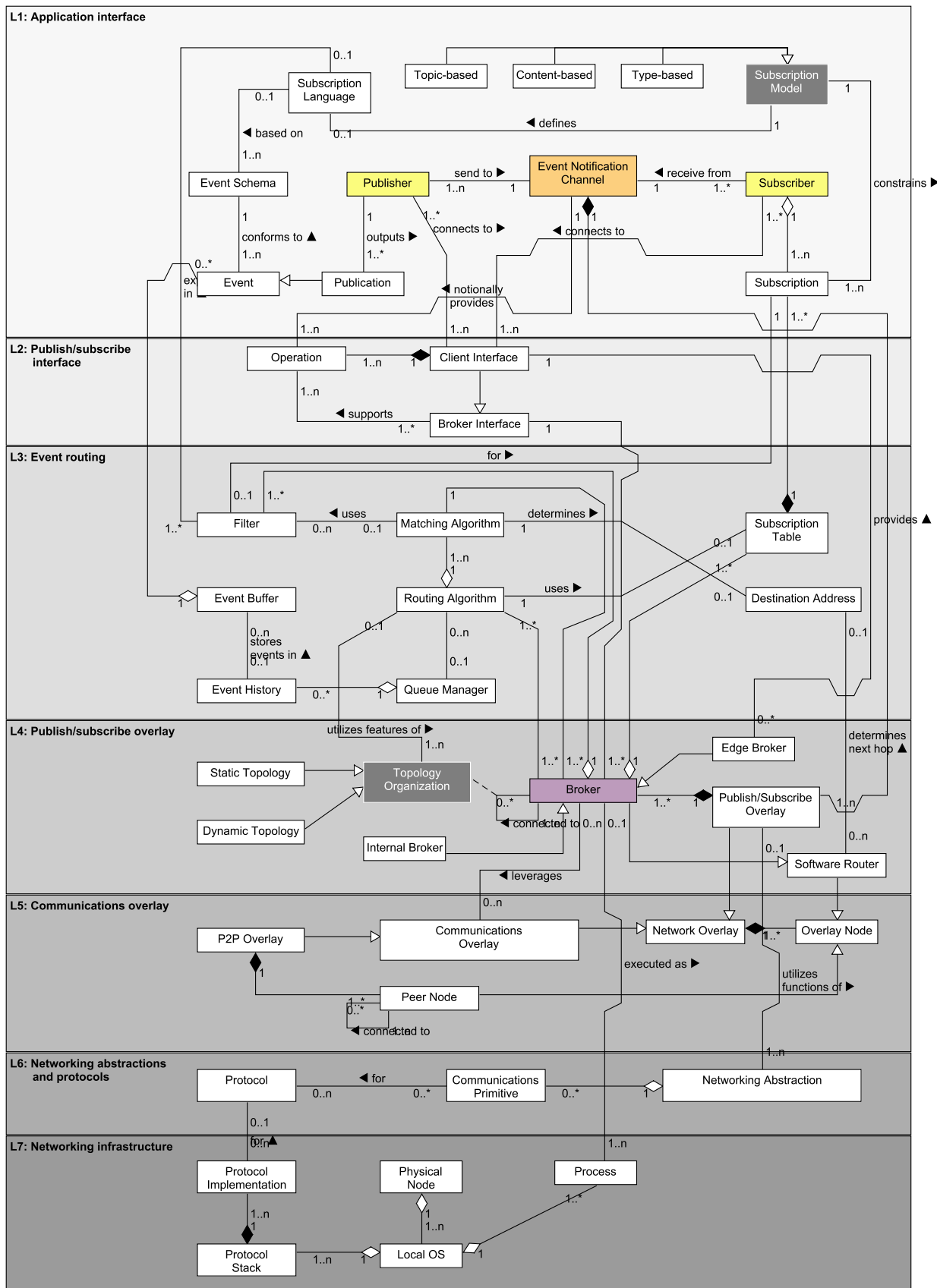
Fig. 1 shows a semi-formal, conceptual reference model inspired by the existing models of Virgillito (2003), Blanco et al. (2008), Baldoni et al. (2009), Hummer et al. (2012), and Tarkoma (2013), which captures the main elements and possible organizations of a software architecture based on the publish/subscribe pattern. Looked at as a whole, the reference model is stratified into seven levels according to the Layers pattern (Buschmann et al., 2007), each one representing a higher degree of abstraction with two (non-strict) dependencies: with respect to implementation and progressing upwards through the levels, upper levels rely on the functionality encompassed by lower levels; and, with respect to architectural design decisions and progressing downwards, decisions made at the higher levels influence those made at the lower levels.

In this paper we use the reference model to provide a more detailed overview of publish/subscribe systems and hence to introduce consistent terminology and concepts for our solution reviews in Sections 4 and 5; and also to provide architectural contextualization when defining relevant security threats in Section 3.2. Of course, it is not difficult to envisage other uses of the model, which are outside the scope of this paper: for example, incorporating the model as an essential conceptual security artefact in a system-specific security methodology (see Section 7).

We should emphasize that our reference model is not intended to replace the models from which it takes its inspiration or the numerous detailed taxonomies of publish/subscribe systems – and their more general variants – in the literature (see Section 2.1), both of which are required for a deeper understanding of the area; rather, its value lies in complementing the latter work and promoting a unified understanding of publish/subscribe systems at a high-level of abstraction. Similarly, our reference model – as indeed other related informal and semi-formal models – is not a replacement for formal computational models, such as can be found in the work of Baldoni et al. (2005) or Mühl et al. (2006).

In what follows, we describe the reference model and its main elements, organizing the presentation around the model's layered structure.

3.1.2.1. L1: application interface. The top-most level of our reference model gives a notional and application-level view,



containing what is essentially the representative model of the original publish/subscribe pattern which can be found in, for example [Buschmann et al. \(2007\)](#) or [Riehle \(2011\)](#). As the top three model elements suggest, any instance of the publish/subscribe pattern has three essential participants: publishers (producers of events), which generate data; subscribers (consumers of events), which receive the generated data according to interest; and a logical event notification channel or service, which performs the actual data dissemination by acting as a dynamic application-level router ([Huang and Garcia-Molina, 2004](#)). From an application-level perspective, the notification channel is a “black box” with an interface, such that publishers publish events and subscribers receive events.

Besides the three main elements of the publish/subscribe pattern, the next most important element is the *subscription model*, which also refers to one of the two main characteristics used to distinguish publish/subscribe systems ([Bellavista et al., 2014](#)) (the second one is considered below in the exposition of L4). The subscription model of a publish/subscribe system determines the overall specification of events and affects the manner in which they are routed within the event channel (see L3 below). The major models are (following [Corsaro et al., 2006](#); [Eugster, 2007](#); [Eugster et al., 2003](#); [Liu and Plale, 2003](#)):

- *Topic-based*: where events have locally or globally unique identifiers (usually identifiable character strings such as “command-stream”), sometimes organized hierarchically to allow granular sub-specification (e.g. “command-stream.actuators”, “command-stream.”) and thereby to preclude redundancy in the receipt of events.
- *Content-based*: where the contents of events, usually in the form of sets of value–attribute pairs, determine routing decisions, and where corresponding subscriptions are specified via a dedicated filtering language (e.g. XPath or a custom language). The advantage of the content-based model over the topic-based model is that subscribers have greater ability in determining the events received ([Mühl et al., 2006](#); [Segall et al., 2000](#)).
- *Type-based*: where events are correlated to object-oriented types, thereby implying type-safety and methods as well as attributes ([Virgillio, 2003](#); [Eugster, 2007](#)).

3.1.2.2. L2: publish/subscribe interface. In practice, applications need to connect or be linked in some fashion to realizations of publisher and subscriber components – thus the latter components need to provide application interfaces, which may well be wrappers that in turn interface with actual publisher/subscriber functionality. These interfaces are not the only ones required: in general, the event notification channel must also provide external interfaces to publishers and subscribers, offering an API for the relevant functionality ([Pietzuch et al., 2007](#)). This interface may be consistent across all components within the publish/subscribe overlay level (L4); or provided by a certain subset of components (either from the “standard” set or one or more specialized components). In all cases, the main operations provided by the interfaces are: *publish* (event), *subscribe* (class of events), *unsubscribe* (class of events), *notify* (event); and optionally *advertise* (class of events) and *unadvertise* (class of events).

3.1.2.3. L3: event routing. The event routing layer encompasses the core functions of the notification channel, which includes the routing and matching algorithms (see [Fig. 1](#)). Events passing through the notification channel are matched to given subscriptions via filters and, depending on the routing algorithm used, forwarded to nearby elements within the structure of the channel (see L4 below), or delivered directly to interested subscribers. This implies that the publish/subscribe pattern supports undirected communication semantics, whereby the address of any given destination component is determined dynamically ([Pallickara and Fox, 2003](#); [Sutton et al., 2001](#)). As there is also no need for direct interaction between the publishers and subscribers, publish/subscribe can be regarded as a specialization or distinct form of the Implicit Invocation pattern for distributed object interaction ([Avgeriou and Zdun, 2005](#)).

There are a number of valid routing algorithms that can be utilized at this level, with the majority falling into the categories of selective filtering (where each subscription is filtered at some point in the notification channel’s structure, requiring a subscription or routing table); gossiping (a probabilistic neighbour forwarding approach); flooding (where events are broadcast through the channel); and rendezvous (where a particular node acts as a routing point for some class of events) ([Baldoni et al., 2009](#); [Martins and Duarte, 2010](#)). As [Baldoni et al. \(2009\)](#) point out, routing algorithms exploit the structure of the publish/subscribe overlay for event delivery, which is described next as part of the fourth level of the reference model (L4).

3.1.2.4. L4: publish/subscribe overlay. At this level the event notification channel is no longer seen as a “black box” component, but a virtual (application-level) network or overlay of brokers ([Mühl et al., 2006](#)) – the software routers referred to previously in the description of L1. This overlay can be structured in various ways, and is often cited as one of the main characterizing features of publish/subscribe systems, named variously the *event channel’s architectural realization* or *topology organization* ([Bellavista et al., 2014](#); [Mayer et al., 2012](#)). The main organizations are (following [Pesonen, 2008](#); [Tarkoma, 2009](#)):

- *Centralized fixed topology*: a single broker acts as the centralized event routing/forwarding component for the whole system, thus implying a single point of failure, but also increased ease of implementation.
- *Distributed static topology* (sometimes called *hybrid peer-to-peer* or *partially decentralized*): a static topology implies a graph-like distribution of brokers that is predictable and not expected to change significantly over time, with the major varieties being hierarchical (e.g. tree-based as in JEDI), acyclic (where event-flows passing through brokers are not allowed to form cycles, e.g. REBECA); cyclic (event-flows form a general graph); or a combination of the above (e.g. as in SIENA – see [Tarkoma and Raatikainen \(2006\)](#) for references).
- *Distributed dynamic topology* (sometimes called *pure peer-to-peer* or *fully decentralized*): in a dynamic topology the publish/subscribe broker overlay relies on a secondary (genuine) peer-to-peer overlay or is incorporated into such an overlay (see L5 below), which can have structured or unstructured peer-to-peer semantics.

When speaking of distributed publish/subscribe systems (the subject systems of this paper), we imply systems with architectures whose overlay topology organization falls into the last two categories.

3.1.2.5. L5: communications overlay. As pointed out above, architectures using the dynamic topology overlay organization model generally require a peer-to-peer overlay, such as Pastry or CHORD. An example of a system with such an architecture is Scribe (Castro et al., 2002). It is also conceivable, however, that the publish/subscribe overlay may use some other type of application-level overlay for different communications patterns.

3.1.2.6. L6: networking abstractions and protocols. The upper levels of the reference model all rely on networking functionality – e.g. multicast, broadcast, messaging middleware, object-based middleware (such as CORBA), or communication frameworks (such as ACE) (Schmidt et al., 2000), which are presented to upper layers via various networking abstractions. In terms of protocols, multicast in particular can be used for topic-based systems (see L1), as in these cases a one-to-one correspondence can be formed between topics and groups of publishers/subscribers (Baldoni et al., 2009); however, this communication approach is mostly limited to LANs where suitable protocols can be applied.²

3.1.2.7. L7: networking infrastructure. The lowest level of the reference model contains protocol implementations such as TCP or UDP, and all the low-level network functionality present in the underlying operating systems at each distributed node.

3.1.2.8. Realizing model elements. Naturally, the details of each of the elements in the reference model can themselves be realized using numerous other patterns, both for general networked and distributed systems (Buschmann et al., 2007; Schmidt et al., 2000) as well as event-based systems more generally (Paschke et al., 2012). In fact, there are essentially no limitations on what patterns can be used – the internals of the broker components can even be realized using separate instances of the publish/subscribe pattern itself (Tarkoma, 2013).

3.1.3. Types of publish/subscribe systems

Instances of the publish/subscribe pattern need not be limited to providing the basic communications/interaction functionality mandated by the pattern's semantics in the form of a *publish/subscribe infrastructure*; it is also possible for a given system to augment the basic functionality with additional services supporting various distributed system transparencies while retaining the publish/subscribe-centric focus, in which case one can speak of *publish/subscribe middleware* (Pietzuch, 2002) – cf. (Schmidt et al., 2000; Carvalho et al., 2004; Voelter et al., 2005). The literature from the last fifteen years presents over forty concrete examples of both classes of systems, ranging from standardized middleware developed by industry (CORBA EventService, RTI DDS) to research-driven systems from academia, variously exploring routing algorithms (JEDI,

REBECA), fault tolerance (TERA), scalability (SIENA), reconfiguration (REDS, GREEN), custom policy enforcement etc. The features of these systems, and indeed the publish/subscribe pattern in general, are a core element of more elaborate or more specialized distributed event-based systems (Cristea et al., 2011; Hummer et al., 2012), which are also sometimes termed “publish/subscribe systems”. While the latter definition would be more general and encompassing, in this paper the term “publish/subscribe system” will be used in a narrower sense to mean infrastructures and middleware focusing on providing the core functionality encapsulated in the publish/subscribe pattern.

3.2. Security threats for publish/subscribe systems – an abstract threat model

Publish/subscribe systems are susceptible to a wide range of security threats at both the network and host levels, which have been discussed in the past by various authors whenever delineating an explicit threat model for a given set of security solutions, via *ad hoc* analysis, and using systematic threat enumeration (Wang et al., 2002; Pesonen et al., 2007; Solum et al., 2009; Altaweel, 2009; Tariq et al., 2010; Fernandez and Uzunov, 2012; Ion et al., 2012). A sound knowledge of such system-specific threats is crucial for the determination of appropriate security requirements (Möckel and Abdallah, 2010; Myagmar et al., 2005) – indeed, security requirements can be specified solely and directly in terms of threats (see Uzunov et al., 2015a). To this end, in Table 1 we present a collection of publish/subscribe system-specific threats based on a “light-weight” application of the threat patterns (particular kinds of abstract software patterns) for general distributed systems of (Uzunov and Fernandez, 2014). The term “light-weight” is understood here in the sense that we do not aspire to create a complete publish/subscribe system-specific taxonomy, but merely to distil a wide range of threats, as threat patterns, from both the literature and by specializing existing patterns to a distributed publish/subscribe system context, characterized more concretely with the aid of the conceptual reference model of the previous section. We term the result an *abstract threat model*, inasmuch as it is a threat model that does not relate to a particular system, but to a class of systems, i.e. to distributed publish/subscribe systems in general. As can be inferred from the foregoing, the model is not exhaustive (even if it does cover most or all documented threats in the publish/subscribe literature to date), but encompasses a comprehensive range of threats, and is representative of the major threat categories affecting publish/subscribe systems³ – something that can be confirmed when the model is considered in the context of

² Notwithstanding IP multicast, which can be difficult to implement Internet-wide (Tarkoma, 2013).

³ It should be noted that the threat patterns in the abstract threat model pertain to publish/subscribe systems predominantly at the L1 to L4 layers in Fig. 1. The infrastructure underlying a publish/subscribe overlay, e.g. a peer-to-peer communications overlay at L5, may well be susceptible to additional threats (for which one could consider the peer-to-peer threat taxonomy of Uzunov and Fernandez (2014), for example). Similarly, concrete-system-specific features such as caching of publications at routers (cf. Abdallah et al., 2015) would require the consideration of additional threats, which cannot, however, be captured in an abstract threat model for general publish/subscribe systems.

Table 1 – Publish/subscribe system-specific threats.

TN	Threat name	Threat pattern(s) specialized	Architectural contexts	Threat description	A	Mitigating high-level security policies
T1	Event eavesdropping (external actor)	Message secrecy violation	Communication (message channels); L1, L4, L5	Malicious external actors intercept and read sensitive event contents	S	Secure communication (message encryption)
T2	Event eavesdropping (broker)	Message secrecy violation, Data inference	Communication (networking infrastructure, protocols); L3, L4	Malicious internal (overlay) brokers read sensitive event contents – stored or in-transit	S	Secure communication (message encryption), Authorization, Storage security (encryption)
T3	Curious subscriber	Message secrecy violation, Unauthorized access	Communication (messages), Data/Storage management (data structures); L1, L2, L3	Subscribers receive and read parts of sensitive events which they should not be able to	S	Secure communication, Authorization, Identity management, Storage security
T4	Eager subscriber	Message integrity violation, Session hijacking	Communication (events, message channels); L1, L2, L3	Malicious entities change in-transit subscriptions in order to receive events outside their subscription scope	I	Secure communication, Authorization, Identity management
T5	Event corruption (external actor)	Message integrity violation	Communication (message channels); L1, L4, L5	Malicious external actors intercept and modify sensitive event contents	I	Secure communication
T6	Event corruption (broker)	Message integrity violation	Communication (networking infrastructure); L3, L4	Malicious internal (overlay) brokers modify sensitive event contents	I	Secure communication
T7	Corrupt broker	Route poisoning	Addressing (protocols, routing data structures); L3, L4	Malicious internal (overlay) brokers modify routes	I	Identity management, Secure communication (secure routing)
T8	Overlay scanning	Scanning, Data inference	Communication (message channels, protocols, networking infrastructure); L3, L4, L5	Malicious actors infer information about publish/subscribe overlay's (run-time) configuration (Aniello et al., 2014)	S	Filtering, Authorization, Logging and monitoring
T9	Corrupt publisher	Injection, Message integrity violation	Communication (messages); L1, L2, L3	Publishers publish malformed or corrupted events	I	Filtering
T10	Spoofed publisher	Identity spoofing, Message authenticity violation	Communications (messages, message channels), Distribution Control (execution abstraction); L1, L2	Malicious external actors send events through the publish/subscribe overlay (i.e. unauthorized publisher publishes events)	I	Identity management (authentication), Authorization, Filtering, Secure communication (MAC, digital signatures), Authorization
T11	Illegal invocations	Invoking unauthorized operations	Resource management (resources – brokers), Distribution control (software component interfaces, operations – brokers), User interfacing; L1, L2, L4	Malicious external actor or unauthorized internal component illegally calls publish/subscribe operations (e.g. unauthorized publishers, subscribers, or brokers join the publish/subscribe overlay – i.e. illegal join)	I	
T12	Subscription leak	Message secrecy violation, Data inference	Communications (message channels), Data/Storage management (data structures – subscription tables); L1, L2, L3	Malicious publishers, subscribers or external actors read the subscriptions of system entities – stored or in-transit	S	Authorization, Storage security, Secure communications
T13	Identity leak	Output information disclosure, Data inference	User interfacing (all), Communications (message channel, protocols); L1, L2, L3	Publishers find the identities of subscribers, or subscribers find the identity of publishers, when these identities should be hidden	S	Authorization, Identity management, Secure communications
T14	Overlay flooding	Message flooding (caused directly)	Communications (message channels); L2, L3	Malicious publishers/subscribers flood the overlay with events, advertisements or subscriptions	V	Filtering, Secure communication, Logging and monitoring
T15	Oversubscription	Message flooding (caused indirectly), Resource exhaustion	Communications (networking infrastructure), Resource management (resources); L2, L3, L4	Large numbers of malicious subscribers subscribe to all possible events, leading to a high volume of events upon publication	V	Filtering, Secure communication, Logging and monitoring, Execution control (quotas and limits)
T16	Subscription churn	Resource exhaustion	Resource management (protocols) L1, L2, L3	Subscribers subscribe to different events rapidly over time (Esposito and Ciampi, 2015 ; Wun et al., 2007)	V	Execution control (quotas and limits), Logging and monitoring
T17	Lossy broker	Session state poisoning	Communications (message channels), Addressing (protocols); L3, L4	Overlay brokers selectively or randomly drop events	V	Secure communication (secure routing protocols)
T18	Intentional broker crash	Targeted process crashing	Distribution control (execution abstractions, processes); L4, L7	Processes executing overlay brokers are purposefully crashed	V	Execution control (process replication)
T19	Event repudiation	Repudiation	Communication (messages, protocols); L1, L3	Publishers or subscribers deny they have sent or received an event, respectively	CC	Secure communications (digital signatures), Logging and monitoring (logging, auditing)

existing threat models, enumerations, etc., for publish/subscribe systems referred to earlier, and for systems incorporating publish/subscribe functionality in some essential fashion (e.g. Information Centric Networks (Ahlgren et al., 2012; Fotiou et al., 2010b) – see AbdAllah et al., 2015). As with threat taxonomies in general, each of the constituent patterns of the abstract model has to first be instantiated to a concrete threat with respect to a given architectural context to form a concrete threat model, which implies that one can derive more threats than are contained in the model *per se* – a number that increases further when two or more patterns are instantiated to form more complicated or specific threats for a given context.⁴ Henceforth we will use the term “threat” to refer to both the patterns in Table 1 and the corresponding *in situ* pattern instances, wherever this ambivalence does not cause confusion – e.g. we will refer to one or more security patterns addressing one or more “threats”, with the implication being that the latter security patterns (i.e. “security pattern instances”) can address one or more concrete threat pattern instances.

The table containing the abstract threat model is structured as follows. The left-most column lists a threat number, which is used later in Section 6 when discussing and evaluating the reviewed solutions. The second column contains a unique name for each threat. The third column contains the threat pattern which was specialized from Uzunov and Fernandez (2014), and the fourth column contains the corresponding architectural context from Uzunov et al. (2013) augmented by references to the layers of the conceptual model in Fig. 1 defining threat-to-layer mappings (the nature of which is explained further below). The fifth column contains a brief threat description. The sixth column (labelled A) indicates the main security attribute (secrecy – S, integrity – I, availability – V, accountability – CC) which a particular threat affects if it is to be realized as an attack. The final column contains mitigating high-level policies as per Uzunov and Fernandez (2014) and Uzunov et al. (2015a), which are: Authorization, Identity management, Secure communication, Execution control, Storage security, Filtering, and Logging and monitoring. When appropriate, we specify next to the high-level policies one or more specific security policies in brackets. The numbering of threat patterns in the table loosely proceeds in accord with the affected security attributes (A in column 6) – first S, then I, V, and finally CC.

Regarding the mappings in Column 4, it should be noted that while each threat can, in theory, be mapped to every one of the first four reference model layers (L1 to L4) – and especially to L1, inasmuch as L1 captures a notional, high-level view of publish/subscribe systems – some layers are more relevant for a given threat than others, and it is precisely these, more relevant layers that are indicated in the table. By “mapped” in the above sense, we mean that in a concrete architecture a given threat – upon instantiation – can be related to components realizing one or more conceptual elements within the

indicated layers. For example: given some architecture, *Corrupt publisher* (T9) can be mapped to our reference model such that the final threat instance relates to realizations of the *publications* element (L1 – see Fig. 1), made through a realization of the *client interface* element (L2) and routed without proper filtering in one or more *filter* realizations (L3). The latter example reveals another important use of the mappings when designing and/or analyzing security solutions, namely, vertically “passing down” each threat across the layers as is done in the analysis process of (Uzunov et al., 2013) and relating them to elements in (the most) relevant layers, in order to form “chains” of threats and to determine the most affected components.

Regarding the relation between threats and security requirements alluded to earlier, although we do not consider the point further, it is worth mentioning that from the threat list in Table 1 one can easily determine a list of security requirements of the form “enforce X” where X refers to the mitigating security policy. In this way one could make eight general requirements corresponding to the eight high-level policy classes, and refine these as required via more specific policies.

Regarding the actual high-level policies themselves, it should not be too surprising to find that *Secure communications* features quite prominently in Table 1, since publish/subscribe is, after all, a pattern for communications. Nevertheless, we should point out the *Secure communications* (high-level) policy, as we see it, encompasses a rather broad range of more specific (sub-) policies, which can be mapped to an equally broad range of security mechanisms beyond traditional two-party secure channels – including, for example, secure group communication and secure routing mechanisms (Uzunov and Fernandez, 2016). Of course, it is often the case that one of several alternative policies, or indeed several policies together, need to be realized in order to address a given threat pattern instance; hence the presence of *Secure communications* in the last column of Table 1, if other policies are also present there, should not be understood to imply that all threats can, or should, be addressed with only (or at all with) solutions realizing this particular high-level policy.

To further highlight the range of policies that can be used to address the threats from the abstract threat model, we present a “reverse” view of Table 1 in Table 2, this time listing the threats (second column in Table 2, which refers to TN from Table 1) that could be mitigated by particular high-level policy realizations (first column in Table 2, which refers to the last column from Table 1).

Table 2 – High-level security policies and threat patterns that corresponding policy realizations could mitigate.

High-level security policies	Threat patterns that could be mitigated (→TN)
Authorization	T2–T4, T8, T10–T13
Identity management	T3, T4, T7, T10, T13
Secure communication	T1–T7, T10–T15, T17, T19
Filtering	T8, T10, T14, T15
Storage security	T2, T3, T12
Logging and monitoring	T8, T14–T16, T18
Execution control	T15, T16, T18

⁴ For example, the first six threat patterns (T1 to T6) from Table 1 can be instantiated in different combinations to form concrete threats that are realizable as the *Watchlist* and *Sniffing* attacks of (AbdAllah et al., 2015).

Table 3 – Threats grouped according to the threat categories of Uzunov and Fernandez (2014).

Threat classes	Threat patterns
Identity attacks	T10
Network communication attacks	T1–T7, T9, T10, T12, T14, T17
Network protocol attacks	None
Passing illegal data	T9
Stored data attacks	None
Remote information inference	T2, T8, T12, T13
Loss of accountability	T19
Uncontrolled operations	T3, T11, T15, T16, T18

Two final remarks are in order regarding the categorization of threats and the compass of the model as a whole. As can be seen from Table 1, the threats in the abstract threat model are grouped primarily according to the affected security attributes: secrecy (T1 to T3, T8 and T12 to T13 – six in total), integrity (T4 to T7 and T9 to T11 – seven in total), availability (T14 to T18 – five in total) and accountability (T19 – a single threat). The categorization of the threats according to Uzunov and Fernandez’s threat classes for general distributed systems is shown in Table 3. As can be seen from that table, the majority of threats belong to the *Network communications attacks* category, which, again, should not be too surprising given the nature of the publish/subscribe pattern. Table 3 also shows that several threats which are specializations of two patterns are accordingly present in two categories. This apparent inconsistency is perfectly acceptable, since the threat model was not constructed to be a taxonomy in which every threat must belong to only one category.

Another feature worth noting in Table 3 is that none of the patterns from the *Network protocol attacks* and *Stored data attacks* categories were specialized. The reason for this is that, in our view, the (original) patterns in these categories do not need to undergo any specific modifications in a publish/subscribe systems context; and moreover, these categories include threats pertaining to what would be mapped to L6/L7 of the conceptual reference model from Section 3.1.2, hence being outside the model’s compass. This implies, of course – as stated earlier – that the compass of the abstract threat model is a restricted subset of threats affecting general distributed systems; and hence the threat patterns in Table 1 should be seen not as an all-encompassing list, but as representative of *general-system-specific*, but *particular-system-agnostic*, threats affecting distributed publish/subscribe systems predominantly at the L1 to L4 levels with respect to the reference model in Fig. 1, captured abstractly. While in developing a secure publish/subscribe system one should consider the union of all applicable threats (which include those for general distributed systems), in practice the relevant solutions in the literature focus on the subset captured in the abstract model presented in this section; hence we likewise focus on these threats and in effect – for the rest of the paper – on how they have been mitigated.

3.3. Categories of security solutions

A *security solution*, taken in a broad sense, means simply the specification of one or more mechanisms and/or protocols to address a given set of security threats, and hence to satisfy

one or more security requirements – such as one of the requirements stemming from the threats introduced in the previous sub-section – in the context of a particular software system. A security solution can thus be seen (as in the Introduction) as a collection of one or more security pattern instances. This does not imply, of course, that solutions are structured in a “flat” fashion; rather, they can be structured hierarchically, with one solution composed of others.

An observation of the different security solutions proposed by various authors in a publish/subscribe systems context reveals several distinct solution categories. The first of these categories encompasses all solutions that attempt to specifically enforce one of the high-level security policies from Section 3.2 (right-most column of Table 1) via a set of mechanisms or, more often than not, protocols (usually cryptographic in nature) that are system implementation- and architecture-independent. In this case, such solutions are not only *abstract*, but also *isolated*, having no relation to others as they only focus on one particular problem (for example, the enforcement of authorization or secure communication). These solutions can thus aptly be termed *abstract isolated solutions*. A consideration of other security policies leads to a solution composed of numerous (sub)solutions which may be related to one another or unified by some common mechanism, protocol, etc. – i.e. interconnected – thus leading to an *abstract security framework* or *scheme*. To realize an abstract solution of any kind in a concrete system requires that a number of principal design decisions be made – decisions that would have to be made in conjunction with the architectural development of the particular system under consideration, and which carry a degree of dependence on that system’s software architecture. Making such decisions turns the abstract solutions into *concrete solutions*, and in particular into concrete isolated solutions (for corresponding abstract isolated solutions), or into *concrete security architectures* (for corresponding abstract frameworks or schemes, or groups of isolated solutions). Clearly, concrete security architectures consist of a group or an interconnected set of concrete isolated solutions, but in practice one would speak of the “security architecture” of a system, rather than a system having “one isolated solution”, hence, as a category, concrete isolated solutions can be subsumed into concrete security architectures. This implies that there are three major categories of solutions, which for the sake of brevity we term: (abstract) *isolated solutions*, *security frameworks*, and *security architectures*, with the level of abstraction and connectedness implicitly understood.

While the distinction between abstract and concrete solutions is sufficiently sharp in theory, in practice the authors of several systems claim some degree of genericity for their security architectures, and several abstract solutions implicitly imply specific constraints on target systems, making clear categorization occasionally difficult. The general rule that should (and has, in this survey) be applied in such cases is to consider descriptions of solutions in the context of specific software architectures (even abstract architectures in standards) belonging to the first (“concrete”) category; and descriptions of solutions made without reference to a specific architecture (e.g. evaluated by simulation) – even when constraints are placed on the use of the solution – belonging to the second (“abstract”) category.

4. Overview of concrete security solutions – security architectures

The following sub-sections present the core constituent of this survey – a series of reviews of concrete solutions attempting to encompass most of the major distributed publish/subscribe systems possessing nontrivial security architectures. While representative of all the categories of such architectures and comprehensive in coverage, the reviews do not aim to be exhaustive or to include all relevant publications, as several omissions were made mainly due to lack of detail on particular features or strong similarity of features with respect to other systems/solutions already reviewed (e.g. with respect to systems, Scribe and OpenSplice DDS, although significant, belong to one or more of these categories). We have organized the reviews according to a sub-categorization of the *concrete security architectures* category, where each sub-category – except for the last one – captures a major distinguishing characteristic or a differentiating “focus” of a given architecture, as described below:

- *Plugin-based architectures* employ some form of (static) pluggable component mechanism around which security functionality is built.
- *Policy-driven architectures* focus on the enforcement of policies defined in some formalized language, where security policies are explicitly or implicitly regarded as belonging to just one particular policy category among others.
- *Authorization-focused architectures* place pre-eminent emphasis on access control and authorizing entity actions, even though other security requirements are, in general, also considered.
- *Dynamically managed architectures* utilize re-configuration algorithms to change elements of the architecture at run-time.
- *General architectures* attempt to satisfy several security requirements in an *ad hoc* fashion, possessing no particularly distinguishable characteristics.

With respect to presentation, we first review general architectures, and then architectures in the other sub-categories following the order above. Several other architectures that we felt deserved mention (which also fall in the “general architectures” category), but that are not central to our reviews, are described briefly at the end of this whole section.

Since our focus is on the security architectures of the respective systems under consideration, we do not in general describe the actual (functional) system architectures or their features in any significant detail, especially when such detail would be unnecessary for understanding a given system’s security architecture *per se*.

As mentioned in the Introduction, at the end of each review we include a brief analysis and evaluation indicating the main security patterns employed to enforce the high-level security policies relevant to each given solution – as per Table 1 (Section 3.2); along with which threats – from the abstract threat model introduced in Section 3.2 – these patterns seek to address. The analyses also serve the dual purpose of (further) characterizing each solution under review along a “vertical” dimension, as explained earlier. These brief analyses appear in Tables 4–16, which are structured as follows:

- **Column 1 (Security policy):** The first column lists the high-level security policies relevant to the given solution under review.
- **Column 2 (Security patterns):** The second column indicates the (major) security patterns that enforce the latter high-level policies, with the pattern names taken from the literature (Bunke et al., 2012; Fernandez, 2013; Hafiz, 2013; Hafiz et al., 2011; Steel et al., 2005; Uzunov et al., 2012a) (for descriptions of the individual patterns the reader is referred to the latter references). When a given pattern – whether published at the time of this writing or pending publication – is part of a security solution frame structure, we append a mnemonic X in pointed brackets (i.e. “<X>”) where X refers to a pattern family (see Uzunov and Fernandez, 2016; Uzunov et al., 2015a, 2015c), which can be one of: **KM** – key management family; **SC** – secure two-party communication family; **AN** – authentication family; **AZC** – conceptual authorization model family; **AZE** – authorization enforcement architecture family. Of the latter frame-encapsulated patterns, only those are mentioned which are deemed most relevant, instead of a whole pattern hierarchy as would otherwise be done when describing a solution frame application. Patterns in italic font with an asterisk in brackets (i.e. “*Pattern name* (*)”) are neither in the current literature nor known to have been explicitly distilled by anyone, but are sufficiently common among the reviewed solutions to warrant indication. We attempt to briefly summarize the essential content of these (as yet undocumented) patterns in Appendix A.
- **Column 3 (Ch.) and Column 4 (Ds.):** The third and fourth columns, respectively, indicate the characteristics of the pattern instances (Ch.) – abstract (**A**) or concrete (**C**); and the way their realization is described in the publications pertaining to the solution reviewed (Ds.) – actual (**A**), where the relevant descriptions are explicit and with appropriate focus; or implied (**I**), where the relevant pattern instances are only mentioned, receive a brief description in passing, or are simply assumed to be present and/or required by other pattern instances without additional details.
- **Column 5 (Threats addressed):** The final, fifth column lists the threats addressed from Table 1, with “supportive” meaning the threat(s) is/are not directly addressed, but that the pattern instances provide some form of support for the threat(s) to be addressed.

4.1. General architectures

4.1.1. Elvin4 (Table 4)

Elvin4 is an evolution of the Elvin(3) content-based publish/subscribe middleware developed at the University of Queensland (Segall et al., 2000). Elvin4 is primarily based on a centralized server architecture, however, via a specific protocol the system can be extended to encompass server federations (Arnold and Lister, 2007) and thereby to form broker network overlays.

Security is one of the features to have received attention in the Elvin4 evolution process, with emphasis on what the authors term “authorized communications”, i.e. secure communications with coarse-grained authorization. The architectural decisions leading to the realization of the latter

Table 4 – Security patterns and threats addressed in the Elvin4 security architecture.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Authorization	Authorization, Controlled access session, Object mediation/Trusted proxy	C	A	T3, T4, T11
Identity management	Authenticator, Application-driven authentication <AN>, Password-based authentication <AN>	C	A	T10
Secure communications	Secure channel (SSL)	C	A	T1, T4, T5
Security information management	Centralized key management architecture <KM>, Participant-based key management <KM>	C	A	Supportive

Table 5 – Security patterns and threats addressed in the Harmony security architecture.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Authorization	Authorization	A	I	T3, T4, T11
Identity management	Remote authenticator/authorizer (TPM), Credential, Event tagging (*)	C	A	T3, T10
Secure communications	Secure channel	C	A	T1, T4, T5
Security information management	Session key exchange <KM>	C	A	T1, T5

security goals are the introduction of an additional security layer with SSL and Kerberos modules interposed between the transport-level network stack and a data marshalling layer. Publishers use the secure communication mechanisms supplied by the Elvin security layer to transport a collection of keys (as in other security architectures, essentially playing the role of capabilities) to an appropriate server, which stores and matches them against a cryptographic hash of any subscriber-transported keys during event delivery/subscription. Keys can be shared among entities to enforce event scoping (cf. [Section 4.2.1.1](#)), and can also be tied to a server connection (akin to a session) to promote authorization continuity. This scheme clearly relies on the robust management of the publisher/subscriber keys, which is, however, outside of the scope of functionality provided by Elvin4 – keys are expected to be managed at the application level.

In an extension of Elvin to handle mobility, [Arkins \(2000\)](#) and [Sutton et al. \(2001\)](#) introduce intermediate Proxy components into the Elvin broker overlay acting as persistent subscribers with respect to the brokers/servers and as servers with respect to the actual mobile subscribers. Proxies employ sessions to account for, and keep track of, multiple subscribers across several physical devices or applications, which are secured – in relative terms – with a per-user password-based authentication scheme. In both the original and mobility-extended Elvin4, the intermediate brokers (servers) are assumed to be completely trustworthy.

4.1.2. Harmony (Table 5)

A multi-purpose communications infrastructure with a focus on mobility is the Harmony middleware developed at the IBM Thomas J. Watson Research Centre ([Dube et al., 2008](#); [Karenos et al., 2009](#); [Yang et al., 2009](#)), which supports both the mediated, distributed publish/subscribe and unmediated, point-to-point/multicast event-driven models.

Harmony's security architecture relies predominantly on the use of trusted hardware, in particular the Trusted Platform Module (TPM) and the attestation functionality it provides to perform secure session key exchange. Nodes that do not use

the TPM are assigned tags to form separate security domains, which are complementary to the administrative domains to which the nodes belong. In both cases, nodes in domains receive credentials as identifiers, so that events/messages are routed to authorized nodes only (the precise details of the authorization features, as well as the tagging and domain separation, do not appear to be documented in the open literature). Routing is based on overlay nodes dynamically determining the best path based on QoS requirements per security domain, and is one of the main differentiating features of Harmony ([Karenos et al., 2009](#)). With respect to security, however, the main contributions of Harmony's security architecture – as it is described in [Dube et al. \(2008\)](#) – are relatively standard, consisting in the enforcement of message confidentiality via encryption and authenticity semantics, as well as the aforementioned per-domain/tag QoS separation.

4.1.3. NaradaBrokering security framework (Table 6)

As the name implies, the NaradaBrokering security framework⁵ ([Pallickara et al., 2005, 2006](#); [Yan et al., 2003](#)) is based on the NaradaBrokering distribution middleware, which allows several distinct communication styles, including peer-to-peer and publish/subscribe ([Pallickara and Fox, 2003](#)). With respect to its publish/subscribe functionality, NaradaBrokering takes a topic-based approach, and utilizes an overlay structured into a hierarchical sets of broker clusters – separate groups of brokers forming sub-networks between themselves with a controller broker within each group. This architecture allows robust routing via a dynamic, optimal path selection algorithm.

The main goal of the NaradaBrokering security framework is to achieve “secure communications between authorized entities” ([Pallickara et al., 2005](#)), and hence to enforce message

⁵ Although clearly related, the word “framework” here does not refer to our classification of security solution types as per [Section 3.3](#), but is used by the NaradaBrokering authors in a general sense. The precise terminology with respect to our classification should be “NaradaBrokering Security Architecture”, and for this review in particular the reader should assume “framework” has that meaning.

Table 6 – Security patterns and threats addressed in the NaradaBrokering security architecture.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Authorization	Dedicated server <AZE> (KMCs), Credential/Capability	C	A	T3, T4, T11
Identity management	Brokered authentication, Mutual authentication, <i>Event tagging</i> (*), <i>Authentication challenge</i> (*), Public key infrastructure <AN>	C	A	T3, T7, T10
Logging and monitoring	Security logger and auditor (in conjunction with SSL & Mutual auth.)	C	A	T19
Secure communications	Secure channel (SSL), Digital signature with hashing, Trust partitioning (channels), Message replay detection	C	A	T1, T4, T5, T10, T14, T19
Security information management	Centralized key server <KM>, Public key infrastructure <KM>	C	A	T1, T5, T10

confidentiality, integrity and authenticity, along with authentication of entities and authorization for the publish/subscribe operations. To achieve these goals, the security framework incorporates an out-of-band (notionally centralized) certificate authority (CA) and a collection of key management centres (KMCs), each of which internally incorporates an authorization module, within the existing NaradaBrokering architecture. Both publishers and subscribers are given a public key certificate by the CA, which is subsequently used to authorize topic and subscription registrations with a local KMC, the latter operations performed by publishing to a special topic which itself may have restricted access (Pallickara et al., 2008). Of central importance to the security of the communications scheme is the concept of a “secure topic” – conceptually a pair consisting of a topic and an associated symmetric key managed by a KMC instance. Per-topic symmetric keys are distributed to entities via a secure channel, alongside with KMC hashed and signed per-entity tokens (essentially capabilities) packaging an entity’s certificate and rights, which are presented to brokers by both publishers and subscribers when performing each of their associated functions. The use of per-entity tokens supplants the earlier Access Control List based authorization model used in the NaradaBrokering framework (Pallickara et al., 2005, 2007) (cf. (Pesonen et al., 2007)), however, the validation operation that must be performed by brokers in communication with relevant KMCs, and hence their role as points for enforcing authorization, is still necessary. Message integrity is ensured via hashed digital signatures (using the CA-distributed private key in conjunction with RC2 (Yan et al., 2003) or SHA-1 (Pallickara et al., 2006) by default), while confidentiality is achieved by encrypting events with the per-topic symmetric key. SSL is used in broker-to-KMC communications, ensuring end-to-end protection of the message exchange between publishers and receiving entities.

In contrast to the other security architectures described thus far, the NaradaBrokering framework includes specific preven-

tative measures for several classes of attacks, including event replay – via garbage collected message IDs (Pallickara et al., 2007) or timestamp storage (Pallickara et al., 2006); event repudiation and man-in-the-middle attacks – via SSL and mutual authentication during per-topic key distribution (Pallickara et al., 2006); as well as denial of service attacks – by detecting compromised topics (periodic authentication challenges) and KMC isolation (indirect communications through specific topics) (Pallickara et al., 2006).

Like RTT’s Connex DDS, the NaradaBrokering security framework allows for per-topic access control, this time via the use of per-topic shared keys, which are obtained from the KMC.

The NaradaBrokering security framework provides a comprehensive basis for a secure publish/subscribe system, employing the defence-in-depth principle to enforce authenticated, authorized and secure communications. The main weakness of the framework lies in its relative centralization. In particular, if a link to a KMC is broken, then the security features are adversely affected to the degree that all secure topics managed by that KMC become unusable.

4.1.4. Sec-SDDL (Table 7)

Sec-SDDL (Secure SDDL – Scalable Data Distribution Layer) (Gonçalves et al., 2013) is a security-enhanced version of SDDL, a DDS-based communications infrastructure used in the ContextNet pervasive, mobile systems project (Endler et al., 2011). Sec-SDDL is based on the DDS standard (more precisely, the DDS RTPS protocol is used for real-time communication within a core fixed/wired network), together with mobility enhancements, for which another protocol, MR-UDP, is employed, the details being outlined in Gonçalves et al. (2013). Events in Sec-SDDL are routed within the core wired network, or through Gateways to mobile nodes.

The main aim of the Sec-SDDL security architecture is to realize secure communications with event/message integrity, confidentiality and authenticity semantics (via encryption

Table 7 – Security patterns and threats addressed in the Sec-SDDL security architecture.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Authentication	Certification infrastructure <AN>, Brokered authentication, Certificate-based authentication <AN>, Single sign on	C	A	T3, T4, T10
Secure communications	Secure channel, Digital signatures with hashing, <i>Event tagging</i> (*)	C	A	T1, T4, T5, T19
Security information management	Centralized key management architecture <KM>, Centralized key server <KM>, Centralized keying protocol <KM>, Session key exchange <KM>	C	A	T1, T5

and digital signatures), as well as publisher/subscriber authentication (via a certificate-based authentication scheme) while retaining scalability, introducing as little overhead as possible, and permitting independent/per-topic security-feature enforcement. Authorization is not considered – plans for future work do include, however, the implementation of a context-aware, spatio-temporal conceptual authorization model in line with the aims of the ContextNet project.

The Sec-SDDL architecture itself relies on two centralized components, both of which directly address the secondary problems arising from the use of cryptography to realize message confidentiality and integrity, and the primary problem of ensuring node/publisher/subscriber authenticity: a centralized certificate authority (CA) and a centralized key distribution centre (KDC). Digital certificates are used for identity establishment and validation, for which a Kerberos-like protocol is employed, with tickets being used to provide time-limited single-sign on semantics, thereby obviating the need to verify the publisher/subscriber's certificate for each request. As is usual for such authentication protocols, a side effect is the generation of symmetric session keys (one per topic, periodically replaced), which is subsequently used for secure communications. Tags are used as identifiers for channels, to allow for different topics to be secured independently.

With respect to implementation, the latter security functionality is realized as a separate layer in both fixed and mobile nodes, which initiates the authentication and keying protocols; manages the local key store; and performs the necessary cryptographic functions locally, for each node. The solution was evaluated via simulation and shown to be scalable to several thousand nodes with little delay.

4.2. Plugin-based architectures

4.2.1. REBECA (Table 8)

REBECA (Mühl et al., 2006) is a content-based publish/subscribe system originally developed at the University of Darmstadt. Some of REBECA's salient characteristics are optimized routing algorithms for reducing overhead in an acyclic static topology (Baldoni et al., 2009; Mühl et al., 2006) as well as dynamic re-configuration and self-organization (Jaeger et al., 2007; Parzyjega, 2014; Parzyjega et al., 2008).

4.2.1.1. *Scoping* (Fiege et al.). The original version of REBECA does not provide any security features, however, early extensions by Fiege et al. (2004) in the context of event scoping (for which see Mühl et al., 2006) introduce basic publisher/subscriber access control. This security feature is realized via public-key attribute certificates (certified by an off-line certificate authority), which are verified at border brokers. While Fiege et al. do not discuss their underlying authorization model in depth, it is reasonable to assume it is a form of ABAC (attribute-based

access control), where attribute certificates are treated as capability realizations and the use of scoping adds primitive group-based semantics – i.e. only nodes in a given scope have access to a given set of notifications. The creation and destruction of the scopes require manual administration, which can be authorized individually using attribute certificates in the same manner as the publish/subscribe operations themselves. Among the more hypothetical extensions discussed by Fiege et al. is that nodes joining scopes at run-time can initiate the enforcement of additional relevant policies; however, this is not developed further.

4.2.1.2. *Encryption via plugins* (Parzyjega et al.). More recently, Parzyjega and colleagues have re-designed REBECA to use a plugin-based architecture (Parzyjega et al., 2010) in response to the proliferation of implementation variants requiring different functional and non-functional features. In the new design and implementation, plugins are provided for core features such as notification transport and matching, as well as for dynamic reconfiguration (optimal path routing, fault tolerance) and adaptation. In all cases the plugins are inserted into the brokers and clients at the message input, output and processing stages as Invocation Interceptors (Voelter et al., 2005) to enable the relevant functionality. Regarding security features, a plugin is provided for basic secure communications, which cryptographically transforms messages (encryption, and one assumes hashing for integrity) as they are sent/received (input/output stages) with per-connection algorithms. While the scoping features of the previous version of REBECA have been carried forward in the form of a scoping plugin, it is not clear whether scoping as such continues to be used for access control enforcement.

4.2.2. DDS security/RTT's Connex DDS (Table 9)

The Data Distribution Service (DDS) is a standardized abstract architecture – currently managed by OMG (OMG, 2014b) – for real-time, distributed data dissemination middleware. DDS advocates a data-driven, topic-based publish/subscribe paradigm with additional content- and type-based features. Entities exist within global data spaces (domains), and are divided into domain-level publishers and subscribers, which in turn aggregate data writers and readers, respectively (a relatively comprehensive conceptual model of DDS is presented in Corsaro et al. (2006), and in condensed form in Lu et al. (2008)). DDS advocates an implicit peer-to-peer overlay architecture codified in the RTPS (Real-Time Publish Subscribe) protocol (OMG, 2014c). As of this writing, the DDS standard does not explicitly outline any particular security features, other than the use of QoS parameters which, at least in theory, allow applications to define their own security policies. RTT's Connex DDS middleware does, however, provide a comprehensive set of security features, including identity control, authenticity, event

Table 8 – Security patterns and threats addressed in the REBECA security architecture.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Authorization	ABAC, Credential, Trust partitioning (scopes)	C	A	T3, T4
Secure communications	Secure channel	C	A	T1, T5

Table 9 – Security patterns and threats addressed in the RTI Connex DDS security architecture with “default” plugins.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Authorization	Access matrix, Credential	C	I	T3, T4, T11
Identity management	Direct authentication, Mutual authentication, Event tagging (*)	C	I	T3, T10
Logging and monitoring	Security logger and auditor	C	I	T19
Secure communications	Secure channel, Symmetric encryption, Digital signature with hashing	C	I	T1, T4, T5, T10, T14, T19
Filtering	Firewall (application-level, events)	C	A	T9, T10

confidentiality and event security filtering (RTI Sec, 2013). These efforts by RTI have spurred the drafting of a proposal of an additional standard: DDS Security (OMG, 2014a) – the draft being proposed by RTI and PrismTech, along with support from eProsima and MITRE. The proposed standard essentially stipulates a reference security architecture for DDS, the main features of which we outline below, and view it as an abstraction of RTI’s own middleware (Connex DDS Secure) security architecture.

The core of the DDS Security architecture consists of five plugins – extensible, interchangeable security components for authentication (Authentication Service plugin, providing one-way and mutual authentication algorithms); cryptographic functions (Cryptographic Service plugin, providing interfaces for key generation and exchange, encryption, message authentication codes, hashing and digital signatures); event logging (Logging Service plugin); security label attachment (Data Tagging plugin); and authorization on publish/subscribe as well as middleware-specific operations (Access Control Service plugin, supporting the basic Access Matrix model) (DDS Security, 2014). Every entity joining a DDS domain is required to authenticate, and afterwards can only communicate with entities whose identities are similarly established. The authentication process can also generate an initial shared secret (i.e. a key), which can be used by the Cryptographic Service plugin to establish and distribute subsequent cryptographic keys among participating entities, e.g. for secure communications. Support for secure communications is similarly supported by the Cryptographic plugin, which provides an API for applications to enforce message confidentiality, integrity, authenticity and non-repudiation. Entities that join a domain to publish and/or subscribe to certain topics are immediately authenticated, thereby receiving an identity token, and subsequently authorized based on the received token. The same process is applied following peer entity discovery. The actual details of the security functions, and indeed the implementation of the different plugins, are application and middleware specific.

RTI’s own Connex DDS, which we assume for the sake of this review to be either fully compliant, or equivalently so, with the proposed DDS Security standard, realizes and complements the features described above with process separation via MILS (multiple independent levels of security); inter-domain bridging and secure routing; TLS and DTLS protocols for secure intra-domain communications; and message inspection/filtering (RTI, 2012). The specifics of the latter (additional) features are not currently documented in the open literature.

The proposed DDS Security standard, and RTI’s Connex DDS seen as a realization of this standard, at first appears to boast a comprehensive security feature-set addressing a broad range of threats, with a high degree of flexibility through the utilization of a plugin architecture; however, not all the details are specified, and the actual realization – notwithstanding the default implementation included in RTI’s products – is vendor-specific. In particular, a number of details of Connex DDS’s security architecture are not openly documented (perhaps understandably so, as Connex DDS is a proprietary system), which makes assessment difficult. In Table 6 we highlight the security patterns realized with respect to the “default”, RTI-supplied Connex DDS security plugins.

4.3. Policy-driven architectures

4.3.1. PADRES (Table 10)

PADRES is a static topology, content-based publish/subscribe middleware with a focus on Enterprise application features (Jacobsen et al., 2010; Li et al., 2010). According to the University of Toronto’s MSRG (Middleware Systems Research Group) project website (<http://www.msrg.org/projects/padres/features/security.php>), PADRES officially supports transparent secure routing, as well as provisions for tolerating DoS attacks within the broker overlay (in-progress at the time of this writing). Although not intended to be a security feature as such, PADRES

Table 10 – Security patterns and threats addressed in the PADRES security architecture.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Authorization	Authorization, Interceptor <AZC>, Trust <AZC>	A	I	T3, T4
Identity management	Authenticator	A	I	T3, T4, T10
Filtering	Firewall (application level, events)	A	I	T9, T10
Secure communications	Secure routing (*), Secure channel	C	A	T1, T5; T2, T6 and T7 for secure key distribution
Security information management	Decentralized key management architecture <KM>	C	A	T1 and T5 (Supportive); T2, T6 and T7 for secure key distribution
General system security	Policy, Policy delegate, Policy enforcement point	C	A	Supportive

also implements network-based monitoring and administration, which is important in configuring any security controls and also in supporting a limited form of anomalous behaviour detection.

4.3.1.1. Secure routing/share-based key management (Yoon). PADRES has been used as a basis for addressing the problem of reliable and secure key distribution in the work of (Yoon, 2013). As part of a solution for confidential communications built on top of PADRES, Yoon (2013) proposes the use of a (k,n) -secret sharing scheme for key distribution in the presence of Byzantine brokers – i.e. brokers with arbitrary or malicious behaviour. The scheme relies on distributing the shares of the cryptographic keys over a set of dynamically replicated brokers, whereby the broker replication can be controlled according to pre-set (configurable) reliability and performance parameters. Broker replicas are built on-the-fly, and notionally constitute single step routing points (“virtual nodes”), over which the k key shares, initially split at the publisher’s end, are distributed. A certain degree of fault tolerance is also required to ensure that the keys are distributed correctly, which reaches an upper limit set by the number of colluding Byzantine brokers in each virtual node. The advantage of Yoon’s scheme is that all the security functions are performed by existing brokers (replicas), obviating the need for additional infrastructure.

4.3.1.2. Custom policy enforcement (Wun and Jacobsen). PADRES also serves as a reference implementation for the policy-driven framework of Wun and Jacobsen (2007), which is designed for content-based publish/subscribe systems utilizing static topology overlays. Wun and Jacobsen’s proposal allows ECA (Event-Condition-Action) policies to be specified using a simple language for border brokers (event ingress/egress) and between brokers (routing) either prior to or after content-based event matching, with the latter being emphasized to reduce duplication of processing. Policies can either be attached to events – and hence saved in brokers – or associated with a given broker. Wun and Jacobsen demonstrate policies for identity verification leading to basic authorization; security zoning (essentially scoping) via rule-defined trust groups; and firewall filtering, which extends the content-based filtering capabilities of the underlying publish/subscribe system (in this case PADRES). However, the details of the security implementation, including the protocols, algorithms and even the actual mechanisms, are left unspecified and hence for the application developers to realize on their own.

4.3.2. ESCAPE (Table 11)

Taking the idea of custom policy enforcement as a central focus, Russello et al. (2011) propose a policy-driven, publish/subscribe middleware and associated tool-support for wireless sensor

network (WSN) based sense-and-react applications, in the form of ESCAPE – Event-State-Action Policy Environment. ESCAPE – which refers more precisely to the policy management framework for Russello et al.’s middleware, however, we use the acronym here loosely to refer to the whole infrastructure – is founded on the principles of strict separation of concerns, realized via components, which implement an application’s core functionality, as well as core middleware functions; and policies, which govern non-functional features and platform-specific configuration. The policy-driven approach can apply arbitrary policies using the Event-State-Condition-Action policy language, and hence specific security policies (in the spirit of Sloman’s (1994) policy-driven approach to distributed systems) which, like all ESCAPE policies, are seen as re-usable units.

Architecturally, ESCAPE is realized as a topic-based publish/subscribe communication layer with a static topology. Much of the core publish/subscribe functionality is encapsulated in separate components in a Services/Extensions library, including the routing algorithm and communication semantics, both of which are interchangeable at run-time. To enforce the policies, Policy Enforcement Points (PEPs) are placed between applications and (edge) brokers of the publish/subscribe overlay, as well as between the individual overlay brokers. These PEPs play the role of Invocation Interceptors (Voelter et al., 2005), enforcing via a Policy Manager security and other non-functional properties (including communication and pub/sub semantics) independently of the application code, limited by the availability of relevant components in the middleware Extensions/Services Library. In its policy enforcement model, ESCAPE effectively takes an aspect-oriented approach, whereby the policies are “woven” as units of implementation that are notionally transformed into executable code, which strongly differentiates the middleware from other publish/subscribe systems covered in this survey.

Despite the arbitrary potential of ESCAPE, the current collection of built-in components allows for the definition of confidentiality and integrity properties only via encryption and authentication. Example policies that have been specified by Russello et al. include tamper protection and secure group communication policies, with the latter requiring an initial manual/out-of-band shared key distribution (for each WSN node).

4.4. Authorization-focused architectures

4.4.1. Gryphon (Table 12)

Gryphon is an early content-based publish/subscribe system developed at the IBM T.J. Watson Research Centre (Banavar et al., 1999). One of the central innovations in Gryphon is the concept of an *information flow graph* – a “virtual” acyclic event representation used for history-based routing that can be mapped to different physical topologies. According to IBM (2014) and Liu and Plale (2003), the Gryphon security architecture incor-

Table 11 – Security patterns and threats addressed in the ESCAPE security architecture.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Secure communications	Secure channel, Message tampering protection <SC>, <i>Secure group message channel</i> (*)	C	I	T1, T5
General system security	Policy, Policy delegate	C	A	Supportive

Table 12 – Security patterns and threats addressed in the Gryphon security architecture.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Identity management	Direct authentication, Application-level authentication <AN>, Password-based authentication <AN>, Certificate-based authentication <AN>	C	I	T3, T4, T10
Authorization	Authorization, Access matrix, ACL	C	A	T3, T4, T11
Secure communications	Secure channel	A	I	T1, T4, T5

porates several authentication methods, both weak (password-based) and strong (using SSL and digital certificates), as well as support for different “secrecy and integrity policies for different parts of the information space” (IBM, 2014) in the information flow graph.

4.4.1.1. Dynamic access control architecture of Zhao and Sturman. Zhao and Sturman (2006) present an access control architecture for Gryphon which takes into account the system’s specific redundant-path routing protocols while allowing dynamic rule updates. The authorization model used in Zhao and Sturman’s proposal is based on the Access Matrix, with additional constraints on subject/object pairs expressed as content-based (attribute) filters, and the ability to specify groups of subjects. Rights – consisting of connection, publication, subscription and advertisement – are granted to principals, which are verified identities in a one-to-many relationship with publish/subscribe clients. The rule structure is realized in the form of Access Control Lists (ACLs) that are cached at each node in the overlay and stored in a per-domain centralized database with associated version numbers.

Authorization rule updates by a system administrator are effected in an atomic fashion, in the manner of a “batch commit” operation, and subsequently propagated throughout the system. As the versions of the rules are locally cached on overlay nodes, it is possible to guarantee that all events prior to an authorization rule update are processed according to the old rules (in a given message stream, at a given overlay node, according to the information flow graph), and that all new events following the same update are processed according to the new rules. Besides the rules themselves, appropriately restricted content filters and subscriptions are propagated through the overlay. This optimization ensures that new authorization rules are enforced not only at the point of publication, but also progressively at the intermediate overlay nodes, as well

as at the receiving nodes that the final subscribers are connected to.

4.4.2. HERMES extensions (Table 13)

HERMES, first introduced in the work of Pietzuch (2002, 2004) is a type-based publish/subscribe middleware with a dynamic topology, peer-to-peer overlay architecture. Several attempts have been made over the years to introduce various security features into HERMES (or using HERMES as a reference implementation), with particular emphasis on authorization using an extended Role Based Access Control (RBAC) model. Below we discuss several of these attempts in chronological order with a relatively higher degree of detail than the other reviews so far, not only because of the greater availability of information on the subject, but also because the HERMES-related architectures directly attempt to address many of the threats in our threat model from Section 3.2.

4.4.2.1. RBAC security architecture of Belokosztolszki et al. The work of Belokosztolszki et al. (2003) was the first to introduce a comprehensive security architecture for realizing extended RBAC in HERMES, and also one of the earliest examples of introducing any security features into a publish/subscribe system in general. The RBAC implementation is based on OASIS (Open Architecture for Secure Internetworking Services) – see (Belokosztolszki et al., 2003; Bacon et al., 2005) for references.

A central premise of Belokosztolszki et al.’s approach to access control is that only authorized entities can use the system, which implies that all publishers and subscribers must present credentials prior to performing any operations. Two types of credentials can be used for this purpose: digital certificates, which necessarily imply authentication; or capabilities, which may encompass identification/naming on a group or individual entity basis, or may simply be used as tokens that are

Table 13 – Security patterns and threats addressed in the security architecture of HERMES/MAIA inc. DIFC extensions.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Identity management	Certification infrastructure <AN>, Dedicated server <AN>, Brokered authentication, Certificate-based authentication <AN>, Credential, Credential delegation	C	A	T3, T4, T7, T10
Authorization	RBAC, Capability (SPKI cert), Delegatable rights <AZC>, Asymmetric encryption, Reified reference monitor, Controlled access session, Trust partitioning	C	A	T3, T4, T10, T11
Secure communications	Secure channel (SSL), Digital signatures with hashing, Symmetric encryption with Obfuscated transfer object (event attributes), Pseudonymous identity (event type instances)	C	A	T1, T4, T5, T17, T19
Execution control	Controlled execution domain, Event tagging (*)	C	I	T2, T6
Security information management	Decentralized keying protocol <KM>	C	I	T1, T5

possessed by a given entity. The actual authorization functionality is implemented at the edge overlay nodes to which publishers/subscribers are connected, and which are explicitly trusted. The aforementioned edge nodes enforce authorization decisions according to application-specific rules whether a publication (determined via advertisements) or subscription is allowed; this is done only once, after which restrictions are applied. Several predicates are supported for these restrictions, including generic non-optimized restrictions, which are independent of the publish/subscribe infrastructure (i.e. can apply arbitrary policies); and publish/subscribe restrictions, which correlate with the underlying publish/subscribe event filters. Hybrid predicates are also possible.

The inherent trust requirements for the overlay nodes can be reduced by the use of event scoping (cf. the work of Fiege et al. (2004) – Section 4.2.1.1): a trusted set of brokers enforces a protected path for a given event type. Certificate chains implemented using X.509 digital certificates – with the initial certificate signed by a given event type owner – are used so that each broker in a given (scoped) path can verify the authenticity of other brokers.

With respect to the realization of the RBAC model specifically, three roles are defined by default: “publisher”, “subscriber”, or “typeowner”. Authorization policies are specified by event type owners, however, it is not clear whether the typeowner role can be used in the specification of meta-authorization policies. The majority of relevant authorization rules are stored at the overlay nodes (more specifically, a rendezvous node for a given event type), but certain rules are also in a centralized repository.

4.4.2.2. Multi-domain extensions of Bacon et al. In Bacon et al. (2005), the original architecture of Belokosztolszki et al. is extended to a multi-domain setting, instigating the inclusion of selective, per-attribute encryption, trust separation between brokers and basic key management algorithms as an essential part of the architecture’s definition.

Bacon et al.’s extended security architecture supports the latter functionality by including a per-domain, centralized OASIS server that not only serves as a policy management point, but also as an authentication and key distribution server. Subscribers and overlay nodes all possess asymmetric key pairs bound to X.509 certificates, which are used for PKI-based authentication and allow for overlay nodes in particular to form trusted groups, i.e. a more well-defined version of the scopes used in Belokosztolszki et al. The key pairs are also used to establish secure communication channels between the system’s entities, which use the TLS protocol for implementation. Both attributes of publications and subscription filters can be encrypted selectively at end-point nodes (local to the publisher/subscribers) with a separate, per-domain symmetric key. The management of these keys occurs only between overlay nodes via group-based algorithms (Zou et al., 2010) – at the time of writing Bacon and colleagues were investigating the use of several potential algorithms.

With respect to the authorization functionality, the policy language used in Bacon et al.’s RBAC realization allows for the expression of parameterized roles, as well as environmental conditions (e.g. time, date, or for a given event attribute).

4.4.2.3. MAIA/capability-based security architecture of Pesonen, Bacon, et al. MAIA is a derivative of HERMES developed by Pesonen (2008) specifically as a basis for research into secure events and capability-based access control (Pesonen and Bacon, 2005; Pesonen et al., 2006, 2007). Indeed, it is precisely secure event types and the use of capabilities, in the form of SPKI certificates, that differentiates the security architecture proposed by Pesonen et al. from the earlier work of Belokosztolszki et al. and Bacon et al. discussed in Sections 4.4.2.1 and 4.4.2.3 above. By secure event types Pesonen et al. mean guaranteeing the authenticity and integrity of event type instances via the use of an asymmetric cryptographic scheme. Since in HERMES all event types have an owner, the solution used is to digitally sign a given event type instance with the owner’s private key and to include the type owner’s public key as part of the type instance meta-information (extended in MAIA from HERMES). The resulting signed event instances are thus verifiable using the encapsulated public key. To allow updates on event types, the type owner can delegate its public key via an SPKI certificate (signed capability) to another node in the overlay, which becomes a type manager. Verification subsequently takes the form of checking certificate chains, as in related certificate-based security architectures – see, for example, (Chakrabarti, 2007; Xiao, 2007). The use of UUIDs as well as the digital signatures ensures the global uniqueness of event types and hence prevents bogus duplicates. Event type names (instances) are also hashed using SHA-1 for routing across intermediate overlay nodes, providing an additional level of defence-in-depth (relative event type anonymity) against eavesdropping and selective event dropping.

Signed capabilities in the form of SPKI certificates are used not only in the context of managing Pesonen et al.’s secure types, but also as a ubiquitous means for realizing the OASIS RBAC model. As in the previous security architectures reviewed above, authorization policies are enforced for joining the broker overlay (overlay nodes) and attaching to a given local node (publishers and subscribers); for creation and extension of event types; as well as all for publish/subscribe operations offered by the local node – i.e. the publish/subscribe API – for a given event type. Two sets of signed capabilities are used to handle these cases: authorization capabilities, which encapsulate the core authorization rights, such as the right to subscribe to a given event type; and authority delegation capabilities, which grant the right to issue authorization capabilities to entities in the system. The second of these is used by domain controllers, which are specific nodes responsible for access to the broker overlay (both with respect to functionality and joining the overlay), as well as for the creation of new event types; while the first set of capabilities is used by per-domain centralized authorization servers (“access control managers”), which also manage local authorization policies just as in the architecture of Belokosztolszki et al. The highest level capabilities are delivered to nodes out-of-band.

The last prominent feature of the MAIA security architecture, already present in at least a germane form in its predecessors, is attribute encryption via per-attribute symmetric keys. Such attribute encryption functionality, including in its scope the encryption of corresponding subscription filters to ensure privacy, is seen in the context of enforcing fine-

grained access control, allowing for diminished trust in the broker overlay as only those parts of events for which a given broker has a key can be accessed (Pesonen et al., 2007). The algorithm used to implement this is relatively non-standard: AES-EAX, where EAX is a special mode that guarantees secrecy as well as integrity via encryption and MAC generation. The per-attribute keys are distributed via the OFT group key-management algorithm (Zou et al., 2010), where overlay nodes are grouped together according to the possession of appropriate group-key-join capabilities. As in the preceding architectures, Pesonen et al.'s solution relies on boundary security, which implies a reliance on trustworthy edge-nodes to perform the security functionality on behalf of applications (Bacon et al., 2008).

4.4.2.4. *DIFC extensions of Shand et al.* Shand et al. (2011), combine some of the earlier work of Pesonen et al., Bacon et al. and Belokosztolszki et al. outlined above, with the policy-driven, distributed information flow control (DIFC) approach found in Migliavacca et al.'s DEFCon-POLICY – a more general event-based system (Migliavacca et al., 2010a) – to propose an end-to-end security architecture that further reduces the trust requirements on HERMES' overlay nodes. This is achieved by creating DIFC tags as a result of run-time activation of roles, in the OASIS terminology, which are subsequently enforced locally by run-time trusted software components on each overlay node and also at the application endpoints. As per the DIFC authorization model – which is a Mandatory Access Control model – this guarantees the full confidentiality of events as well as their integrity, although the latter to a lesser degree as the events traverse the system. DIFC also permits formal reasoning to be applied to the policies. The details of the DIFC model, its adaptation to event-based systems, and its application to a specific, general event-based system can be found in Migliavacca et al. (2010a; 2010b).

4.4.3. PostgreSQL-PS: information-flow/policy-based security architecture of Singh et al. (Table 14)

As in the preceding security architectures for HERMES (and, subsequently, for MAIA), the work of Singh et al. focuses on the introduction of access control in a type-based publish/subscribe system (Singh et al., 2008, 2011). The proposed security architectures is, however, quite distinct. Even though Singh et al.'s work originates from the same research group, HERMES is no longer used as the basis of the system architecture, but rather, the somewhat unusual choice of an extended PostgreSQL DBMS called PostgreSQL-PS (Eyers et al., 2010). The access control scheme used by Singh et al. under the name of Interaction Control is based on a highly customized conceptual authorization model stemming from the flexibility demands

of health-care, for which “vanilla” Access Matrix derivatives are inadequate (cf. Uzunov et al., 2015c).

One of the main characteristics of the Interaction Control model is that all authorization rules refer to locally connected entities, i.e. the model provides “point-to-point” security, where an event, or the flow of events, passing through the publish/subscribe overlay is transformed and restricted, respectively, according to the local authorization rule-set. The Interaction Control rules thus take the form of either *restrictions* or *transformations* (following Singh et al., 2008): restrictions can be applied to either (1) subscriptions (in general) or (2) publication (event) dissemination, in both cases limiting the respective functionality to authorized entities; while transformations – which can be arbitrary, including even event type changes – can be applied to events on a contextual basis. Singh et al. (2011) later extend the latter rule typology to encompass general conditions in addition to transformations, now seen as *event control rules*; and introduce a separation between restrictions for connections to the overlay and for publications/subscriptions. However, the semantics of the two main groups of rule types remains essentially unchanged. Besides the inclusion of context, Interaction Control rules can predicate roles, particular entities' credentials, event content, external data, current and future overlay node connections and other parameters (Singh et al., 2008, 2011), allowing precise specification of need-to-know (effectively per-entity least-privilege) policies.

For architectural enforcement of the Interaction Control model, Singh et al. utilize a pull-based pattern (Uzunov et al., 2015c) where the authorization decisions are enforced locally at each overlay node, which in the PostgreSQL-PS implementation is simply a database with publish/subscribe broker functionality. All authorization rules are specified using XML and, after appropriate conversion, stored as data rows in relational tables at each (local) overlay node. This also allows rule updates via standard publish/subscribe events. Rule conflicts are accounted for via specific detection and resolution strategies (for which see Singh et al. (2008; 2011)). During run-time, authorization rules are converted into secondary publish/subscribe rules specified in SQL-like syntax for publication, notification and subscription, and checked accordingly for authorization enforcement. Rules that imply restriction can give rise to additional filters, which are dynamically incorporated into the overlay nodes and similarly checked during event routing.

The focus of Interaction Control is limited to controlling data access (from a subscriber viewpoint), which in turn can be seen as controlling data dissemination (from a publisher/overlay node viewpoint). The integrity of the data being accessed or disseminated is not considered, although one assumes it is possible to specify sufficiently elaborate policies to restrict the

Table 14 – Security patterns and threats addressed in the PostgreSQL-PS security architecture.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Identity management	Authenticator, Password-based authentication	C	I	T10, T11
Authorization	Authorization, RBAC, Context <AZC>, Attributes <AZC>, Fine granularity <AZC>, Policy enforcement point, Pull authorization strategy <AZE>, Policy language <AZE> (XML), Conflict resolution strategy <AZE>	C	A	T3, T4, T11

modification of data as it passes through the publish/subscribe overlay. Secure communication is similarly not treated as part of Singh et al.'s work using PostgreSQL-PS.

Singh et al.'s more recent work (Singh et al., 2014) applies a similar policy-driven approach in the context of SBUS, a dynamically re-configurable, component-based middleware capable of providing publish/subscribe semantics. Since the emphasis is on general policy enforcement and publish/subscribe security is not a central focus, we omit the details.

Besides authorization, PostgreSQL-PS enforces basic (password-based) authentication when new database (broker) instances join the publish/subscribe overlay (Eyers et al., 2010). The precise mechanisms are not described in detail.

4.5. Dynamically managed architectures

4.5.1. GridStat (Table 15)

Solum (2007; 2009) and Chakravarthy et al. (2010) propose a security architecture for the GridStat publish/subscribe system for electricity/power grids (Gjermundrød et al., 2009; Hauser et al., 2008), which aims to provide a stable communications infrastructure capable of guaranteeing real-time event delivery. GridStat follows the topic-based paradigm, where a single topic is a status variable, and events are electricity/power grid status updates.

Architecturally, GridStat is divided into two inter-related parts: a fixed topology publish/subscribe overlay optimized for real-time communications (*data plane*); and a hierarchical collection of brokers (*management plane*) that configure and manage the QoS of the data plane. The management plane hierarchy differentiates internal brokers residing at a given depth away from a root broker, and leaf brokers residing at the bottom of a given hierarchical branch, which manage QoS policies for a bounded collection of data management overlay nodes termed a “cloud”. Publish/subscribe functionality spans both the management and the data planes: publishers and subscribers invoke management plane functions to publish and subscribe, respectively, with the latter operation implying optimized configuration of the data plane overlay (e.g. filling routing tables in individual nodes); and the actual delivery of events occurs within the data plane. Publish/subscribe invocations can also carry certain QoS parameters that exploit the hierarchical nature of the management plane brokers, e.g. policies can be made according to domain structures.

The GridStat security architecture is based on the concept of run-time interchangeable security modules, which provide

the mechanisms for a given security policy. Structurally, the security architecture mirrors the management plane organization, with a hierarchical ordering of Security Management (SM) servers (akin to the QoS management brokers) that collectively make up a collocated security *management plane*. The hierarchy is differentiated into leaf SM servers, which are directly connected – like their management counterparts – with a “cloud” of data plane brokers (indeed, the same “clouds” as the leaf QoS brokers); as well as internal SM servers – all of which manage security policies in a cascading fashion. Besides policies for each data plane “cloud”; there is also a general group security policy at the root of the SM plane hierarchy applicable to all entities in the data plane. The security policies are specified using XML, and are stored in individual databases linked to each SM server.

On publication and subscription, a security policy is partially initialized and sent to a local (leaf) SM server, which in turn passes the policy through the SM hierarchy to an appropriate SM server managing publications or subscriptions (for the respective operation) in a given “cloud”; verifies the invoker's credentials (see below); and generates the remaining part of the policy. The latter policy is then passed back to the invoking component, which can either download or use cached versions of relevant security modules during run-time. The modules, which are stored in a separate database connected to the relevant SM servers, are thus assigned on a per-topic basis, according to specified publication and subscription policies.

The collection of security modules outlined in the relevant GridStat publications (viz. those cited at the outset of this section) provide most major cryptographic functionality to ensure secure communications – specialized key generators, modules for encryption, hashing and authenticity via digital signatures – as well as basic data filtering (in addition to the overlay event filtering), and are applicable to the data plane, the management plane and the SM plane itself. A data obfuscation module to be used in conjunction with the encryption module is also provided; however, as the use of an appropriate block cipher mode (e.g. CBC) is more appropriate than custom-made solutions for the same problem, its utility appears limited. A specific security protocol, formally evaluated using BAN (Burrows/Abadi/Needham) logic in the work of Chakravarthy et al. (2010), is used for changing modules dynamically, in case a module is found to be vulnerable after deployment. In this respect the architecture strongly resembles the extended REBECA plugin architecture of Parzyjegl

Table 15 – Security patterns and threats addressed in the GridStat security architecture.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Identity management	Mutual authentication, Credential	C	A	T3, T4, T7, T10
Authorization	Access matrix, ACL, Policy, Policy enforcement point, Credential	C	A	T3, T4, T11
Secure communications	Secure channel, Symmetric encryption, Digital signatures with hashing	C	I	T1, T4, T5, T19
Execution control	Controlled object factory (modules), Dynamic service management	C	A	Supportive
Filtering	Firewall (application level, events)	C	I	T9, T14
Security information management	Decentralized key management architecture <KM>, Decentralized keying protocol <KM>, Centralized policy manager <PM> (several instances), Pull security information <PM>	C	A	Supportive

et al. reviewed in [Section 4.2.1.2](#). All module functionality is activated by a *Transmuter* component internal to both the SM servers and the publishers/subscribers.

Besides interchangeable modules, another prominent feature of the GridStat security architecture is its unique key management strategy. As with other security features, the key management functionality is integrated into the SM plane, with a manual bootstrapping process requiring a collection of pre-set shared keys (key pad) to be installed on each new node partaking in the system. A security protocol updates keys periodically, and also when new nodes are added, by globally incrementing to the next available shared key (key switch), working in conjunction with a separate protocol that attempts to obviate man-in-the-middle attacks that would otherwise disrupt the key synchronization process ([Chakravarthy et al., 2010](#)). A default key generator inside the SM server is provided in addition to specific key generation components included as part of the security modules. To protect both the keys and modules, an elementary ACL-based access control mechanism is incorporated into the SM servers for each data plane “cloud”, controlling access on a per-topic, per-publisher/subscriber basis. [Chakravarthy et al. \(2010\)](#) propose a strong mutual authentication protocol based on Diffie–Hellman key exchange between SM servers and also between (bottom-most) SM servers and end-point nodes (i.e. publishers/subscribers), which provides the necessary identity verification prior to authorization enforcement.

Although detailed and fine-grained, the work of Solum et al. does not provide full end-to-end security guarantees, since the overlay brokers are assumed to be trusted components. Initial efforts towards addressing this gap can be seen in the work of [Dionysiou et al., \(2008\)](#), which proposes a novel trust management system called Hestia. Hestia’s architecture is both interesting and comprehensive, but it is neither implemented nor evaluated, the latter being left for future work – hence we will not consider it further here. [Solum \(2007\)](#) briefly mentions a prototypical integration of TrustBuilder ([Lee et al., 2009](#)) in an earlier version of GridStat, seen as an improvement of the ACL-based authorization mechanism in the SM server; however, the integration does not appear to have been retained in later GridStat versions.

4.5.2. GEMOM ([Table 16](#))

GEMOM ([Abie, 2009; Abie et al., 2008; Savola and Abie, 2009b; Savola et al., 2010](#)) is a self-adaptive, topic-based publish/subscribe middleware developed as a collaborative European Union project, with a focus on applicability in business system scenarios. Self-adaptation in GEMOM refers particularly to the handling of faults for increased availability (under the banner of self-healing), with an extended notion of faults that also encompasses security-related incidents (cf. [Mayer et al., 2012](#)) such as DoS attacks ([Paganelli et al., 2011; Wang et al., 2010a](#)), as well as to the situation-specific, run-time modification of security policies and mechanisms ([Abie et al., 2009](#)).

Architecturally, GEMOM is stratified into two levels, in a way reminiscent of GridStat: an *operational* level, consisting of the publish/subscribe overlay and associated components, collectively termed GEMOM nodes or simply *G nodes*; and a *managerial* level, consisting of components that administer and monitor the G nodes, and effect appropriate local and/or system-wide changes as required – i.e. components concerned with adaptation *per se*. This two-level stratification of the architecture perfectly accords with the general architectural style of self-adaptive systems as outlined in [Weyns et al. \(2012\)](#), with the managerial level acting as a “meta-level” with respect to the operational, “system” level. Topics in GEMOM are notionally contained in namespaces, both of which are managed by one or a subset of brokers in the publish/subscribe overlay. Judging by their use in case studies (e.g. [Paganelli et al., 2011](#)), brokers are organized in a static topology.

As one may expect, GEMOM’s security architecture follows the two-fold functional architecture stratification closely, with authentication and authorization G nodes placed in the operational level, and nodes specifically dealing with security adaptation, auditing and logging, as well as security monitoring, placed in the managerial level. A key management and an identity management framework are used to grant identity certificates containing an asymmetric key pair to each system component (the mechanisms used for this, as well as the PKI architecture, are not defined in any of the relevant publications), which can subsequently be used for digital signing and encryption ([Abie, 2009](#)). The use of certificates also facilitates mutual authentication between publish/subscribe brokers connecting to the overlay, TLS/SSL routing, and access control for

Table 16 – Security patterns and threats addressed in the GEMOM security architecture.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Identity management	Certification infrastructure <AN>, Certificate-based authentication <AN>, Mutual authentication, Credential	C	I	T3, T4, T10
Authorization	Authorization (Access matrix), RBAC, Trust <AZC>, Fine granularity <AZC>	C	I	T3, T4
Execution control	Replicated system, Standby, Load balancer, Dynamic service management	C	A	T14, T18, <i>Supportive</i>
Secure communications	Secure channel (TLS/SSL), Asymmetric encryption, Digital signature with hashing	C	I	T1, T4, T5, T19
Storage security	Encrypted storage	A	I	T2
Logging and monitoring	Behaviour-based IDS, Security logger and auditor, Trust <AZC>	C	A	T9, T15 (*), T16 (*)
Security information management	Key management scheme <KM>	A	I	<i>Supportive</i>

publishers and subscribers – all of which are seen as enforcing self-protection (see [Yuan et al., 2014](#)) at different levels of granularity. Access control in GEMOM supports both mandatory and discretionary identity-based authorization models (depending as it does directly on the strength of the authentication mechanism), and is applied to objects of various granularity, including messages, publish/subscribe topics, namespaces and even metadata ([Savola and Abie, 2009a](#)), to ensure that only authorized publishers/subscribers can receive protected events. All authorization rules are defined in relation to an abstract notion of *module operation*, and allow the specification of additional (e.g. temporal) constraints.

As indicated earlier, the primary differentiator of GEMOM with respect to other systems reviewed thus far is the ability to adapt, via either dynamic reconfiguration or restructuring, the security mechanisms outlined above and, more generally, GEMOM's "encryption schemes, security protocols, security policies (and) security algorithms" ([Abie, 2009](#); [Savola and Abie, 2009b](#)). At the centre of these self-adaptation capabilities is a *monitoring infrastructure* that utilizes a collection of metrics combined with decision algorithms for determining when a given reconfiguration or restructuring should be effected as the result of some (corresponding) change in the system or its environment ([Savola and Abie, 2009a](#)). GEMOM thus closely follows the MAPE-K self-adaptation pattern as described in [Bruni et al. \(2012\)](#) and as presented in the autonomic/self-adaptive computing literature (cf. [Salehie and Tahvildari, 2009](#)).

GEMOM's monitoring infrastructure – the first core aspect of self-adaptation – is connected directly to the publish/subscribe overlay brokers, allowing for the immediate reception of information; and indirectly, via subscription, to other components, which in turn utilize the publish/subscribe functionality to disseminate relevant notifications. In the latter case components publish to a *measurement namespace*, i.e. a special topic cluster encompassing specific topics for message delays, security alerts, individual G node metadata and other classes of system/contextual information that can be measured by GEMOM's metrics.

GEMOM's security metrics – the second core aspect of self-adaptation referred to above – are created by decomposing six essential requirements as per [Savola and Abie \(2009a\)](#) (secrecy/confidentiality, integrity, availability, authentication, authorization and non-repudiation) into a tree-like structure to arrive at a collection of per-requirement *basic measurable components* (e.g. Confidentiality cryptographic protection algorithm), which are then added linearly in a weighted sum. The latter metrics are used both locally and globally ([Savola and Heinonen, 2011](#)), with the most interesting (global) use being in the context of a trust management scheme that determines a system-wide "trust level" (or more precisely, "trustworthiness level") based on factors such as the number and severity of faults found in the system over time (for a full definition of the trust metric, see [Savola and Heinonen \(2011\)](#)). Diminished global trust levels and associated measures resulting from, e.g. natural depreciation of authentication measures over time, are an essential element in triggering GEMOM's self-adaptation mechanisms.

The third core aspect of self-adaptation – decision algorithms (analysis, planning) – are not described by GEMOM's

authors in any of the relevant publications, hence one can only assume they are of a simple, rule-based nature at the time of this writing.

Besides adaptation, GEMOM places significant emphasis on enforcing system-wide availability (self-healing), which, like the complementary self-adaptation features, is based on the use of metrics and monitoring. The fault-tolerance/self-healing techniques used to enforce availability include – at the operational level – the use of redundant paths, load balancing, and the aggressive mirroring of publish/subscribe brokers, whereby redundant broker copies are spawned and load-balanced once (crash) failure has been detected; and – at the managerial level, in the monitoring infrastructure – redundancy, such as the mirroring of monitoring information, and the activation of election algorithms for when a given monitor/probe component/node fails. Such strategies allow for consistent failover without loss of information, and also facilitate the detection of possible DoS attacks. In the latter case the values of various metrics (e.g. link state, network throughput and others) are regularly checked, and appropriate overlay re-configurations are effected (cf. [Esposito et al., 2013](#)) if there are any signs of degraded performance and/or notifications from mirror nodes that their copies have failed (see [Abie et al., 2010](#); [Wang et al., 2010b](#) for more details). The DoS detection strategy is based on the adaptive re-configuration approach proposed as part of [Bu et al.'s \(2006\)](#) "survivable overlay" ([Abie et al., 2009](#)). More details on the other GEMOM fault-tolerance/self-healing strategies, such as broker mirroring, can be found in [Wang et al. \(2010a, 2010b\)](#).

Thus far it should be possible to see the projected scope of GEMOM's security architecture, and the attempts of its authors to go "a step further" ([Abie et al., 2008](#)) than comparable publish/subscribe systems. There are two other features of this security architecture that deserve mention, and that attempt to take GEMOM "two steps further", so to speak, even though their import is not analysed or evaluated in the relevant publications.

The first of these features is an integrated anomaly-based intrusion detection framework first outlined in the position paper of [Wang and Bigham \(2008\)](#), and later elaborated in [Kanstrén and Savola \(2010\)](#) and [Kanstrén et al. \(2011\)](#). This framework leverages the monitoring components attached to the individual G nodes to provide anomaly-based detection of system faults (again, under the extended notion of a fault) including SQL injection, command injection, directory traversal and several other classes. Detection is achieved by way of machine learning and statistical models that measure predominantly message features (in the technical sense) such as length, token presence and character distribution, as well as correlations of the latter across multiple messages – see [Wang and Bigham \(2008\)](#) for more details and references to some of the techniques used. The emphasis placed on message-level anomalies makes GEMOM's anomaly detection capability appear similar to the filtering functionality of one of the previous versions of GridStat (cf. [Section 4.5.1](#)); however, in GEMOM's case message-level filtering is just one aspect of anomaly detection, with the other being indirect modification – via the standard monitoring/feedback loop – of the system-wide trust level, which in turn can trigger GEMOM's self-adaptation mechanisms as outlined above.

The second of the aforementioned features is the run-time use of a fuzzing tool and other (as yet unimplemented) automated analysis tools for the “pre-emptive and automated checking (of) vulnerabilities” (Abie et al., 2009). As with the anomaly detection framework described above, detected vulnerabilities directly affect GEMOM’s global metrics, which in turn trigger security self-adaptation.

Despite the comprehensiveness of the security architecture, it is worth pointing out that most publications on GEMOM appear to be position papers published *a priori* to the concrete implementation of relevant parts or components and, occasionally, even architectural design, meaning that many important details are missing. There are also few empirical results based on experimentation and few explicitly stated assumptions regarding what is considered “reasonable” with respect to non-functional properties other than security, such as scalability and performance (an exception being in Abie et al. (2010) regarding the fault-tolerance mechanism of monitoring nodes, which “scales to a limit of tens of nodes, which is considered enough for our applications”). The descriptions in Paganelli et al. (2011) indicate that the functional tests performed in the initial GEMOM case studies were of a rudimentary nature – a fact which, when combined with the aforementioned shortcoming regarding GEMOM publications, complicates an external and objective evaluation of the full benefits of GEMOM’s unique security architecture.

4.6. Other publish/subscribe systems

4.6.1. LANES

Security architectures for other publish/subscribe systems and/or architectures that we feel deserve mention, but that we have chosen not to review in detail, include the work of Lagutin et al. (2010) and Visala et al. (2011) on LANES (Visala et al., 2009), an architecture loosely realizing the publish/subscribe pattern with a hybrid push/pull event notification model. The LANES security architecture utilizes certificates for most features, in conjunction with elliptic curve cryptography and a specialized filter infrastructure for event routing that incorporates DoS resilience. Since LANES uses a DHT substrate (Chord) directly for event storage and retrieval in a way that requires explicit subscribe requests for receiving corresponding data, it can be seen as a kind of peer-to-peer content-delivery network, and hence is not a “pure” publish/subscribe system *per se*.

4.6.2. Radio/UNISON

Radio – a topic-based, static overlay topology, publish/subscribe infrastructure forming the bottom-most layer of the UNISON distribution middleware (Docking et al., 2015) – is another system deserving mention. Radio’s distinguishing characteristic is the formation of local mesh topologies separated by proxy components that provide a high degree of routing redundancy. Basic resilience features help to ensure that routing and especially proxy node failures do not partition or destroy the overlay. At the time of this writing, Radio’s security architecture is relatively simple, being focused solely on enforcing standard secure communication channel semantics (secrecy, integrity, authenticity) via AES-256 encryption and digital signatures.

5. Overview of abstract security solutions

In this section we provide a brief overview of a selection of abstract security solutions (i.e. abstract isolated solutions and security frameworks, as per the definitions in Section 3.3), mainly as a “supplement” to the solutions already reviewed in Esposito and Ciampi (2015). Although related, research that does not propose well-defined solutions, but rather formalizations or more theoretical work (e.g. Minami et al., 2009) is, in general, omitted. In accord with our focus on distributed publish/subscribe systems, we have omitted solutions that are for, or in our judgement appear to be suited to, centralized publish/subscribe systems, such as Opyrchal et al. (2007) or P3S (Pal et al., 2012). As with the reviews of concrete security architectures in the previous section, we append a brief analysis and evaluation of each solution in the form of a table (Tables 17–24) following the conventions set out at the beginning of Section 4.

5.1. Abstract security frameworks

The category of abstract security frameworks, like security architectures, can be divided further based on the range of threats addressed, which gives rise to *comprehensive frameworks* (addressing a range of threats, generally five or more) and *partial frameworks* (addressing several threats, in general two or three). Most abstract security frameworks in the literature are, at the time of this writing, partial, with a significant exception being Srivatsa et al.’s EventGuard, reviewed below. The solutions which follow EventGuard are arranged in chronological order.

5.1.1. EventGuard (Srivatsa et al.) (comprehensive) (Table 17)

EventGuard (Srivatsa and Liu, 2005, 2007; Srivatsa et al., 2011) is a comprehensive security framework that seeks to address a wide range of threats for topic- and content-based publish/subscribe systems. The framework is made up of two major inter-related parts: a collection of notional components called *guards*, which abstractly encapsulate security-related functionality for each major publish/subscribe operation (advertise, unadvertise, publish, subscribe, unsubscribe – refer to Section 3.1), as well as for publish/subscribe routing; and an abstract design for a resilient overlay, which exploits redundant paths for probabilistic routing. Architecturally, the guards act as Wrapper Facades (Schmidt et al., 2000) to the underlying publish/subscribe interface, all of which are layered on top of the resilient overlay. It would be theoretically possible to construct the resilient overlay as the highest layer of a hierarchy, thus treating all other overlays as communications overlays; however, the increased level of indirection would in all likelihood reduce performance.

Regarding the first major part of the framework, Srivatsa et al.’s guards utilize a (notional) proxy component for most security-related functionality, called a *trusted meta-service* (MS) – envisaged as being replicated on-demand – whose purpose is to generate keys and per-topic (attribute) cryptographic tokens (event tags); verify identities; create digital signatures; and optionally enforce authorization policies (if any are specified). Prior to being able to publish, publishers are authenticated via

Table 17 – Security patterns and threats addressed in the EventGuard framework.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Identity management	Certification infrastructure <AN>, Certificate-based authentication <AN>	A	A	T10
Authorization	Secure proxy (MS), Object mediation, Capability (implicit – cryptographic key), Policy enforcement point (in MS, optional) with ACL (<i>assumed</i>)	A	A	T3, T10, T11
Secure communications	Symmetric encryption, Asymmetric encryption, Message replay detection, Message tampering protection <SC>, Event tagging (*), Secure routing (*), Pseudonymous identity (topics)	C	A	T1, T2, T3, T4, T5, T6, T7, T9, T12, T14, T17, T18
Security information management	Key generation, Certification authority, Key management scheme <KM>, Trusted third party <KM> (MS – replicated)	C	A	Supportive

MS-generated certificates as part of the advertisement guard functionality. The relevant certificates, along with the associated public keys of the publisher (MS-generated) and the MS itself, are distributed upon invocation of the advertisement operation (to subscribers), and using an “in-built mechanism” (to publishers), which is not described in detail by Srivatsa and colleagues. Besides this bootstrapping of public keys and certificates, during advertisement the MS generates (symmetric) authorization keys associated with each topic attribute, such that only entities possessing these keys can receive the corresponding events and access the (encrypted) event content; as well as an unadvertisement token, allowing “safe” (i.e. authorized) unadvertisement. Authorization keys and unadvertisement tokens – as well as the similarly-purposed unsubscription tokens, referred to below – play the role of first- and second-class capabilities, respectively, whereby first-class capabilities authorize event accesses and second-class capabilities authorize the unadvertise/unsubscribe operation.

During publication and subscription, the MS transforms the relevant messages – via the (publish, subscribe) guard functionality – into tagged, cryptographically protected versions. The tags are designed to pseudonymize topic attribute names, just as was done in HERMES/MAIA (Section 4.4.2.3). “Cryptographic protection”, on the other hand, implies several things. For publishers, events are encrypted using a random encryption key generated by the MS, and digitally signed using a probabilistic ElGamal scheme and the publisher’s private key, thus enforcing secure communication semantics (secrecy, integrity, authenticity, non-repudiation). The event is also tagged such that it becomes routable. For subscribers, cryptographic protection implies that the subscriptions are digitally signed using the MS’ private key and likewise tagged. Subscribers also receive the aforementioned per-topic (attribute) authorization keys as well as an unsubscription token (along with the other, bootstrapping cryptographic material) upon invoking the subscription operation. The cryptographically transformed subscription (filter) can subsequently be deployed across the publish/subscribe overlay.

Only such cryptographically transformed messages as described above are routed through the overlay – a strategy which, in combination with overlay nodes checking the digital signatures of messages via the MS and detecting message duplication via historical timestamp comparison, implies that any directly caused message flooding is contained to at most the nearest neighbours of a given node.

Once authorized subscribers receive a given event, they can extract the random encryption key using their authorization

key and decrypt the event contents. In a content-based extension of EventGuard, the authorization and encryption keys are related hierarchically, whereby encryption keys can be derived from the authorization key if and only if the subscriber’s filter matches the event, i.e. authorization keys for subscribers become tied to filters. A number of keying algorithms are proposed by Srivatsa and colleagues corresponding to various matching approaches – for details, refer to Srivatsa et al. (2011). In all versions of EventGuard, keys are updated periodically according to the concept of an incremental interval-spaced epoch, which reduces the need for high-volume re-keying while avoiding a flood of subscriptions for each new epoch.

The resilient overlay design, EventGuard’s second major part, is used to consolidate the features of the guards described above, by using multi-path routing such that no broker in the overlay is capable of detecting the frequency of the appearance of a given token, thus guaranteeing DoS resilience (random event dropping) and, to a lesser degree, resilience to node failures.

Although an abstract framework, EventGuard does make some strong assumptions regarding the publish/subscribe systems in which it is to be realized – e.g. the use of topic attributes and advertisement semantics for publication.

5.1.2. Minami et al. (partial) (Table 18)

A partial security framework for publish/subscribe systems that perform in-network aggregation (i.e. data aggregation at the overlay nodes, prior to final subscriber delivery) is proposed in Minami et al. (2008), with a focus on enforcing event secrecy, integrity and authenticity in the context of an untrusted overlay. The framework relies on a centralized security manager component that performs similar functions to the security management layers in GridStat (not an altogether surprising fact, given the power grid emphasis in Minami et al.’s work). In particular, this includes the management of keys, ACLs and subscriptions, as well as the run-time generation of routing and aggregation paths, which are sent to individual brokers in the overlay. All entities in the system are assumed to possess some form of identity as well as public key pairs, which in turn allows for the specification of the ACLs for each event (seen as secrecy security policies), contradicting somewhat the loose coupling of publishers and subscribers. A secure communications protocol employing (partially) homomorphic message authentication codes (MACs) and a (k, n) -threshold secret sharing scheme is used to enforce the required secure communication semantics of events that are aggregated and routed through the overlay. Although we have described the solution here in

Table 18 – Security patterns and threats addressed in the security framework of Minami et al.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Identity management	Dedicated server <AN>	A	I	T3, T10
Authorization	Authorization (Access matrix), ACL, Dedicated server <AZE>	C	I	T3, T4
Secure communications	Secure routing (*)	C	A	T1, T2, T3, T5, T6, T12
Security information management	Key management scheme <KM>	A	I	Supportive

Table 19 – Security patterns and threats addressed in the solution of Dini and Lo Duca.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Authorization	Policy, Policy enforcement point (group members), Credential	C	A	T7, T11
Secure communications	Secure channel (TLS)	A	I	T2, T6

Table 20 – Security patterns and threats addressed in the security framework of Bernard et al.

Security Policy	Security patterns	Ch.	Ds.	Threats addressed
Secure communications	Secure message channel <SC>, Secure routing (*)	A	I	T1, T2, T12
Security information management	Decentralized key management architecture <KM>, Decentralized keying protocol <KM>	A	A	Supportive

a more holistic light, many of the necessary details of the framework are presented as assumptions (a significant exception being the aforementioned protocol), largely leaving their realization open-ended.

5.1.3. Dini and Lo Duca (partial) (Table 19)

The framework of Dini and Lo Duca (2009) aims to enforce message secrecy and integrity via the notion of secure broker groups, the realization of which to a large degree follows the established requirements of secure group communications (cf. Judge and Ammar, 2003), namely, that members must be authorized in some fashion upon joining a group; and that the communication between each group member must be protected. In Dini and Lo Duca's approach the trustworthiness of each broker is first established via non-technical means, and all subsequent group join requests are authorized based on an admission policy, where one, several, or all brokers vouch for the joining broker, in the manner of a “web of trust”. Digital certificates are used to ensure that brokers have some form of identifying information and to support TLS-protected communications between brokers, naturally requiring the existence of a public key infrastructure.

5.1.4. Bernard et al.'s privacy framework (partial) (Table 20)
Bernard et al. (2010) propose a partial framework for topic-based systems consisting of a key exchange protocol and the generic specification of two oracles, which are central abstractions in a conceptual model of fully peer-to-peer (in our terminology, dynamic topology) publish/subscribe overlays. The key exchange protocol, SPTest, is based on commutative hash functions and Diffie–Hellman key exchange, and is used to group overlay nodes based on shared interest (topics). The shared secret key also subsequently allows secure communications. The first of the two oracles, addressing the problem

of event dissemination, provides a *Private Forward* operation, which returns a partial list of overlay nodes which a given event can be forwarded to (effectively implementing matching and routing algorithms); while the second, subscription oracle, provides a (confusingly named) *Private Route* operation, which returns a node in the overlay that is capable of managing subscriptions. In both operations subscriptions and publications are not revealed to ensure there is no subscription leakage (i.e. privacy enforcement). This, however, effectively limits the routing algorithm resulting from the forwarding operation to a random one, or at best a gossip-style algorithm. In practice, one can assume that a subscriber will call the subscription oracle's function repeatedly until an appropriate overlay node is found that also shares the same interest (i.e. topic), whereby the SPTest protocol is used to both test for that interest and exchange a shared key. A similar case will hold true for the dissemination oracle. While several implementation possibilities are discussed, many details are not considered and the framework is not evaluated experimentally. Overall, Bernard et al.'s offers an interesting theoretical solution; however, many details still need to be elaborated before it can be applied in practical situations.

5.1.5. Daubert et al. (partial) (Table 21)

In Daubert et al. (2013), the authors propose a partial framework for publish/subscribe anonymity, confidentiality and integrity aiming above all to retain the scalability characteristics of target systems.⁶ The solution consists in creating a secondary overlay above an existing broker network via advertisements and subscriptions that utilize symmetric and

⁶ Since this survey was written, the partial solution of Daubert and colleagues has been extended in the form of AnonPubSub – see Daubert et al. (2016).

Table 21 – Security patterns and threats addressed in the security framework of Daubert et al.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Secure communications	<i>Secure channel</i> (assumed), Symmetric encryption, Asymmetric encryption, Digital signatures with hashing, Pseudonymous identity (hash chains – topics)	C	I	T1, T2, T3, T4, T5, T6, T12, T17, T19
Security info. management	Key management scheme <KM>, Trusted third party <KM>	A	I	Supportive

asymmetric cryptographic schemes and an out-of-band trusted third-party. The assumption is made that the first-level broker overlay already supports secure inter-broker communication links. With respect to structure, the second-level overlay is a union of sub-networks from the base overlay network, with one sub-network for each topic (termed by Daubert et al. an “attribute”). Hash chaining is used to ensure the anonymity of topics. Once the secondary overlay has been established, all future events/notifications for a particular topic are encrypted and digitally signed, thereby ensuring the confidentiality and integrity within and outside the given set of publishers/subscribers. Despite claims of scalability, Daubert et al. do not present any experimental results, making it difficult to assess whether the solution has achieved its initial goals.

5.1.6. Rao et al. (partial) (Table 22)

Rao et al. (2013) propose a solution with a privacy (anonymity, confidentiality and integrity) focus for topic-based publish/subscribe systems, based on a k -subscription-anonymity model which attempts to provide strong guarantees that leaked subscriptions will not directly lead to identification of subscriber interest. The model is realized via *cloaked subscriptions* – an aggregate collection of both valid and false subscriptions for each topic – in addition to the use of encryption for message confidentiality and integrity. The problem of how to best generate cloaked subscriptions in order to guarantee subscription anonymity (i.e. how many, according to the parameter k ; and with respect to which topics), and subsequently, how to reduce the routing overhead generated by the false subscriptions, can be converted into a linear programming (optimization) problem, for which Rao et al. propose a specific (rounding) algorithm. With respect to the architectural realization of this solution, Rao et al. advocate the use of an anonymizer engine – a trusted architectural component that, with respect to subscription and advertisements, acts as a proxy to the publish/subscribe brokers, inserting false subscriptions for each topic during run-time. One can assume that the algorithm for optimizing routing is implemented at each broker in the publish/subscribe overlay.

Secure multi-party computation is suggested as a possibility when subscribers register their interests via filters to avoid collusion between subscribers and the anonymizer engine in determining actual subscriptions; this avoids the need for trusting the anonymizer engine.

5.1.7. Other security frameworks

Other notable security frameworks include that of Khurana and Koleva (2006), which addresses publication and subscription secrecy and integrity, as well as publication authenticity, using digital signing and existing schemes for proxied re-encryption and secure XML document dissemination; Ion et al. (2012), which uses attribute-based encryption (KP-ABE and CP-ABE) together with proxied encryption as per some of the authors' previous work, and an El-Gamal based multi-user searchable encryption scheme to enforce publication and subscription secrecy, to support key management, and to realize access control; and Tariq et al. (2010, 2014), which applies to dynamic topology publish/subscribe overlays and focuses on the enforcement of event secrecy and integrity, (weak) subscription secrecy, and publisher/subscriber authentication by leveraging Identity-Based Encryption (IBE). The isolated solutions constituting these frameworks are reviewed in more detail in Esposito and Ciampi (2015).

5.2. Abstract isolated security solutions

Below we review a very small selection of isolated solutions. The high-level security policies (from Table 1) which they attempt to enforce or support appear in brackets.

5.2.1. Chaabane and Jmaiel (secure communications) (Table 23)

Chaabane and Jmaiel (2009) propose a solution for enforcing event confidentiality in content-based publish/subscribe systems utilizing the Java (Smart) Card Platform (JCP). The aim of the solution is to replace untrustworthy brokers in a publish/

Table 22 – Security patterns and threats addressed in the solution of Rao et al.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Identity management	Anonymity set, Secure proxy (anonymizer)	C	A	T12
Secure communications	<i>Secure routing</i> (*)	A	I	T1, T2, T5, T6, T13

Table 23 – Security patterns and threats addressed in the solution of Chaabane et al.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Secure communications	Asymmetric encryption, Secure channel, <i>Secure routing</i> (*)	A	A	T1, T2, T5, T6, T7

Table 24 – Security patterns and threats addressed in the solution of Hein et al.

Security policy	Security patterns	Ch.	Ds.	Threats addressed
Authorization	ABAC, Conflict resolution strategy <AZE>	A	A	T3, T11 (supportive)

subscribe overlay with trustworthy routing components that are either linked to, or are running on, the JCP. A secure protocol based on an asymmetric cryptographic scheme is used to setup a secure communication channel between the trustworthy components at the edge of the overlay and publishers/subscribers. The resulting channel has the usual confidentiality-preserving semantics, with message encryption and integrity enforced via 3DES and C-MAC, respectively. All routing decisions are taken by the trustworthy components, which decrypt the data prior to executing the chosen routing algorithm. Chaabane and Jmaiel tested their solution by implementing it as an extension to SIENA, but claim that “this solution can also be implemented on other CBPS (content-based publish/subscribe) systems like Gryphon, HERMES and JEDI”. The aforementioned tests show a marked decrease in performance arising from the cryptographic operations, but within what the authors determined as reasonable bounds. The management of the asymmetric keys used in the initial handshake is not discussed, excepting the mention of the use of a Certificate Authority (CA), which one can assume is either a centralized or an off-line component.

5.2.2. Hein et al. (authorization – ABAC policy support) (Table 24)

Hein et al. (2011) propose a set of algorithms for the automated detection and resolution of access control rule conflicts in the context of an ABAC (Attribute-Based Access Control (Sandhu, 2012)) model. An example of such a conflict is when publishers from two distinct domains wish to publish some information exclusively with a different set of allowed subscribers (publisher–publisher conflicts). Architecturally, a Centralized Access Moderator (CAM) component linked to some form of data storage back-end – e.g. a fact database – is utilized to perform the resolution and management of rules, with the actual modification enforcement occurring at the affected publishers and subscribers. The CAM maintains a consistent set of rules, as well as a history of rule updates – including those rejected during conflict detection and resolution – which supports long-term rule management by either allowing rollback to a previously accepted rule configuration, or by the inclusion of previously rejected rules as updates that can become valid under new circumstances. A separate algorithm is proposed specifically for this purpose. While the applicability of Hein et al.’s algorithms is not explicitly specified, they should be suitable for any publish/subscribe system capable of representing type hierarchies in its event model. This is clearly satisfied by type-based publish/subscribe systems (cf. Pietzuch, 2002; Eugster, 2007), but can also apply to hierarchical topic-based systems insofar that sub-topics can be treated as “sub-types” in an abstract sense (cf. Pesonen et al., 2006).

5.2.3. Other isolated solutions

A number of other authors propose isolated solutions for enforcing privacy, understood variously as enforcing secrecy and

integrity (i.e. Secure communication policies) in the context of an untrusted publish/subscribe overlay or, in other words, as the mitigation of some combination of Event eavesdropping, Event corruption and Subscription leak threats (refer to Section 3.2) utilizing a variety of advanced cryptographic schemes, including: commutative encryption (Burdusel, 2011); multiple layer commutative encryption (Shikfa et al., 2009); asymmetric scalar-product preserving encryption (Choi et al., 2010); modified Paillier homomorphic encryption (Nabeel et al., 2013); redundant attribute encryption (Wang et al., 2013); and hybrid encryption in the context of Conditional Oblivious Transfer (Crescenzo et al., 2014). Most of these solutions are covered in (Esposito and Ciampi, 2015), and a number are also briefly summarized in Ion et al. (2012). The same threats are addressed in Maji and Bagchi’s recently proposed vCAPS protocol (Maji and Bagchi, 2012), which aims to avoid some of the shortcomings of the previous solutions by building on ideas from the SIENA system (such as the SIENA Fast Forwarding algorithm and posets of subscriptions at each overlay node) combined with the concept of a routing vector. The work of Barazzutti et al. (2012), although not proposing a solution as such, supports some of the aforementioned advanced cryptographic solutions via a generic Bloom-filter-based pre-filtering stage that exploits covering relations (i.e. subscription containment for content-based systems, akin to topic hierarchies for topic-based systems (Mühl et al., 2006)) to speed up encrypted-content matching.

Other noteworthy isolated solutions that are not mentioned elsewhere include that of Fotiou et al. (2010b), which presents an approach to countering overlay flooding based on ranking publications and subscribers voting whether the received publications were valid or not. The solution requires the existence of an authentication mechanism and unique identification of publications, and also places restrictions on the overlay routing algorithms (rendezvous based). Rahaman et al. (2010) outline a publish/subscribe routing scheme focusing predominantly on confidentiality, in a context highly specific to ontology-based XML Document dissemination. And Nguyen and Huh (2011) present a novel group key management protocol for multicast communications in the context of publish/subscribe-based Cloud Sensor Networks, which builds on a subscription matrix technique and two existing types of keying algorithms.

6. Discussion

Having now reviewed our chosen set of solutions, in Table 25 we indicate to what extent these solutions (table rows) address the threats (table columns) from Section 3.2. The letters and symbols in each table cell should be interpreted as follows: “Y” (Yes) implies that the solution attempts to directly address the threat in some fashion; “P” (Partially) implies a limited attempt to address a given threat under specific circumstances; “Y*”

Table 25 – Threats addressed by reviewed security solutions.

Solution	Threats (Tn → threat n from Table 1)																		
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
Elvin4	Y		Y	Y	Y						Y								
Harmony	Y		Y	Y	Y					Y	Y								
NaradaBrokering	Y		Y	Y	Y		P			Y	Y			Y					Y
Sec-SDDL	Y		Y	Y	Y					Y									Y
REBECA	Y		Y	Y	Y														
DDS Security/RTI Connex DDS	Y		Y	Y	Y				Y	Y	Y			Y					Y
PADRES	Y	P	Y*	Y*	Y	P	P		Y	Y*									
ESCAPE	Y			Y#	Y														
Gryphon	Y*		Y	Y	Y*					Y	Y								Y
HERMES/MAIA	Y	Y	Y	Y	Y	Y	P			Y	Y						P		Y
PostgreSQL-PS			Y	Y						Y	Y								
GridStat	Y		Y	Y	Y		Y		Y	Y	Y			Y					Y
GEMOM	Y		Y	Y	Y				Y	Y	Y			Y	Y#	Y#		Y	Y
EventGuard	Y	Y	Y	Y	Y	Y	Y		Y	Y	P	Y		Y			Y	Y*	Y
Minami et al.	Y	Y	Y	Y	Y	Y				Y		Y							
Bernard et al.	Y	Y										Y							
Daubert et al.	Y	Y*	Y#	Y*	Y	Y*						Y					P		Y
Rao et al.	Y	Y		Y#	Y	Y						Y							
Chaabane and Jmaiel	Y	Y			Y	Y	Y												
Dini and Lo Duca		Y				Y	Y				P								Y
Hein et al.			Y								S								

(Interpolated yes) implies the publications related to a particular solution indicate that a given threat is addressed, but do not provide any detailed descriptions as to how this is done; “Y#” (Extrapolated yes) implies that the solution includes mechanisms that should, at least in theory, enable it to address a given threat; and “S” (Support) implies that the threat is not directly addressed, but the solution includes mechanisms that support sub-solutions which do, or could, address that threat. Blank cells imply that a solution does not address a given threat.

As can be seen from Table 25, with the exception of (external) event eavesdropping and corruption (T1 and T5), abstract and concrete solutions tend to focus on addressing a complementary range of threats: in particular, abstract solutions tend to place greater emphasis on addressing event eavesdropping and corruption at the broker/infrastructure level (T2 and T6), corrupt brokers (T7) and subscription leaks (T12), by enforcing advanced secure communications semantics (e.g. secure routing); whereas concrete solutions tend to place emphasis on addressing curious and eager subscribers (T3 and T4), as well as spoofed publishers and illegal invocations (T10 and T11), by enforcing authorization and authentication (identity management) policies. Regarding concrete solutions, the security architecture of GEMOM clearly addresses the most threats, not least due to its additional monitoring and logging features; while EventGuard addresses the most threats in comparison to other abstract solutions. Interestingly, EventGuard addresses more threats than most concrete solutions as well, which may be a result of the solution being architected “from the ground up”, whereas many of the concrete solutions are architected *post factum* system design. Nevertheless, as noted in the review (Section 5.1.1), EventGuard does make certain assumptions on its target system, which imply, in effect, a set of somewhat stringent architectural constraints.

Another important observation which can be made from the table is that no single reviewed solution addresses all

threats in our threat model. With respect to concrete solutions, the reason for this could be that in certain cases system architects may have judged some threats as posing less risk (in some circumstances) – e.g. T15 and T16; or may have considered that mitigating a given threat would be too costly, e.g. in terms of performance. For the several threats that are conspicuously not addressed in any solution, one could adduce reasons such as lack of prior knowledge (as for T8, which originates from a recently discovered attack by Aniello et al. (2014)), as well as a lack of requirements or contradictory requirements (as for T13, if entity anonymity was not a design priority or if entity anonymity contradicts the need for entity authentication when addressing T10 (Docking et al., 2015)). Several solutions (e.g. PostgreSQL-PS) focus on narrow application domains, and therefore do not aspire to address more than a few relevant threats by design. Nevertheless, the latter observation indicates that, in many cases, security solutions for publish/subscribe systems would need to be extended in some fashion in order to counter a broader spectrum of threats. To this end Esposito and Ciampi (2015) discuss at length possible future directions for isolated solutions, which apply equally to the constituent solutions of the various security architectures. A number of these directions have in fact already been initiated in the security architectures reviewed in this paper: for example, GEMOM provides auditing for non-repudiation, as well as self-monitoring and self-healing capabilities to ensure a fault-tolerant overlay and to support anomaly detection; Sec-SDDL supports single-sign on (SSO) semantics; PostgreSQL-P utilizes a “non-classical”, advanced authorization model; and GridStat attempts to resolve re-keying issues in the context of cyberphysical systems. This is not to say, of course, that further work cannot be done in these same directions.

On a larger scale than isolated solutions, there are a number of valuable cross-cutting features of the reviewed security ar-

chitectures that could also be elaborated. The most prominent features that deserve mention in this respect include:

- *Trust management*: This implies measuring and determining trustworthiness of certain system components, or incorporating trust as an essential part of authorization (see Chapin et al., 2008; Lee et al., 2009). GridStat and GEMOM both incorporate elements of trust management, however, trust can be measured and even defined differently (Hall and McQuay, 2011), allowing for different realizations of trust management.
- *Information flow control*: This implies determining distributed information flows in the system and enforcing mandatory authorization policies to ensure there is no data leakage. Some of the work of Shand, Bacon, Miglilavacca et al. on enforcing information flow control (Section 4.4.2.4) is a first step in this direction.
- *Ubiquitous security self-adaptation*: As Tarkoma (2013) points out, “runtime configuration and adaptation of pub/sub systems is still a research topic, and typically not supported by products with the exception of load balancing and fault-tolerance features”. This statement reverberates the findings of (Mahambre et al., 2007) from 2007, which indicated the lack of secure event adaptation (i.e. the ability to switch security features on or off according to a given context) in a range of then-current systems. As mentioned above, however, GEMOM has already demonstrated run-time security adaptation using a MAPE-K self-adaptation pattern. Other adaptation patterns (Bruni et al., 2012), as well as different ways of realizing the MAPE-K pattern (cf. Weyns et al., 2013) and different ways of realizing adaptation more generally (cf. Tajalli et al., 2010) are also possibilities.
- *Decentralized security*: Although a system may utilize a decentralized topology for its publish/subscribe overlay, the presence of centralized security components makes the system more vulnerable to availability attacks (cf. Section 4.1.3 on NaradaBrokering). The use of security mechanisms suited to decentralized settings – one example being MAIA’s signed capabilities – can help to directly or indirectly mitigate such threats by eliminating the need for, or reducing the reliance on, centralized components. Distributing the functionality of *all* security-related components, including any necessary online Certification Authorities, key servers, authorization servers and others (cf. Marsh and Schneider, 2004) as an essential part of a security solution’s design would be another step in the direction of improving availability.
- *Plugins and dynamic security re-configuration*: Plugin architectures as in Connex DDS and REBECA have a number of advantages, which can be combined with run-time re-configuration to swap security components (as in GridStat). This could be combined with a policy-driven approach, as in Sec-SDDL and the work of Wun and Jacobsen for PADRES, to effect re-configuration based on policies; and/or with self-adaptation to change the components or even policies of the system at run-time.
- *Solution feature combination (static and dynamic)*: More generally, combinations of the features of the different architectures when constructing new security architectures for new publish/subscribe systems, as outlined above,

would be a promising direction in its own right. The development of generic solutions (cf. Section 7 below) would further aid the latter construction process.

7. Conclusion and future directions

In this paper we surveyed the state of the art in security solutions for distributed publish/subscribe systems, with a primary focus on solutions in the form of concrete security architectures. Following our reviews of 13 such solutions (Section 4), as well as brief summaries of 8 abstract solutions (Section 5), we broadly compared the reviewed solution-set with respect to the threats from the abstract threat model introduced in Section 3.2; and also discussed possible future directions for advancing the state of the art, building on the conclusions of Esposito and Ciampi (2015) for isolated solutions. With respect to the latter, our purpose has not been to identify potential avenues for exploration in security techniques (e.g. novel cryptographic algorithms with improved performance) that can be applied to existing publish/subscribe systems, but to review existing solutions in a holistic manner, in order to provide a comprehensive knowledge-base for constructing the security architectures of new (distributed) publish/subscribe systems. Of course, the re-use of existing solutions in this construction process would necessarily require adaptation of the solutions, as many of them are tied – in one way or another – to their respective actual systems or assumptions about potential systems (see Section 3.3). In this respect, distilling the successful designs or, more generally, groups of interconnected elements composing relevant security solutions into (publish/subscribe-specific) security patterns of various levels of granularity (individual, compound etc.) is an important future direction for supporting the engineering of new secure publish/subscribe systems, largely orthogonal to the advancement of existing solutions or the creation of new ones. As the range of security solutions increases over time, and as these solutions are implemented in an increasing number of publish/subscribe systems, the pursuit of the aforementioned direction will become proportionately worthwhile and its outcomes correspondingly valuable. Indeed, the number of systems and range of solutions in this survey – let alone when combined with those from Esposito and Ciampi (2015) – already provide a broad basis for a pattern language for distributed publish/subscribe system security.

In light of the final conclusion of Esposito and Ciampi (2015), which essentially states that assessing the level of security introduced by a given solution would be a promising research direction, it is interesting to note that of all the solutions surveyed in this paper only GEMOM provided some quantitative means of security assessment. Although there are a number of security metrics in the literature that could be used for a similar purpose, it is not always possible, meaningful, or useful, to provide a quantitative security evaluation. Nevertheless, we hold the view that the prevalent evaluations in the publications pertaining to the reviewed solutions in this paper – which invariably consist of some form of performance trade-off analysis – are not adequate in qualifying the strength or value of the corresponding set of security features. Security testing techniques, for example, via the use of software patterns

Table 26 – Brief summaries of undocumented security patterns indicated in the analyses of reviewed solutions.

Security pattern name	Problem	Solution summary
Authentication challenge	In the context of authentication, how can we ascertain that a party partaking in an interaction is not an impostor?	Send a cryptographically transformed byte sequence (challenge) and require the receiver to prove their identity via the possession of a key which can re-transform the sequence. Examples of the aforementioned transformations are symmetric encryption/decryption and keyed hashing.
Event tagging	How can events be identified with respect to the publish/subscribe overlay?	Augment events with one or more pre-defined byte sequences (tags), which are cryptographically protected. These tags help to identify different types of events, or are simply meta-information about a given event that can be used in conjunction with other event information (e.g. content, attributes, etc.) to inform routing decisions at each overlay node.
Secure group message channel	How can several parties in a group structure communicate securely?	Establish a <i>Secure message channel</i> <SC> between each of the participants in the group structure. This pattern is an extension of the standard secure message channel pattern with appropriate group key management protocols to ensure that (1) each group member has a cryptographic key allowing participation; and (2) there are corresponding key-update mechanisms for when entities join/leave the group.
Secure routing (compound pattern)	How can communications among an interconnected network of system elements be protected?	Ensure end-to-end secure communications over a number of hosts via a combination of cryptographic primitives (applying apposite security patterns for this purpose from, e.g. the <i>Secure communications</i> pattern family) and custom protocols (applying, e.g. the <i>Secure protocol design</i> <SC> micro-process pattern). The latter protocols can be fault-tolerant, so as to ensure that failures of communication links and system elements do not adversely affect routing decisions or, more generally, the reliability of the whole system.

(security test patterns (Smith and Williams, 2012), threat patterns (Uzunov and Fernandez, 2014), attack patterns (<http://capec.mitre.org>) and others), would enable a broader and, in some situations, more meaningful evaluation of a given publish/subscribe security architecture than mere performance trade-off analysis. Such techniques could even be made in the context of a structured security methodology specifically tailored for publish/subscribe systems (e.g. centrally employing the abstract threat model presented in this paper; cf. also Uzunov et al., 2012b, 2015b) – applied “in reverse”, so to speak, for the purpose of security analysis – to ensure that all the major threats which a given security architecture (or more generally, solution) is attempting to mitigate are indeed mitigated, at least at the architectural level.

A final point to note is that the reviewed security solutions are, in the vast majority of cases, concerned solely with application-level security, without regard for any underlying infrastructure (usually a commodity operating system) at any given distributed node. The result of this is that trustworthiness is either assumed or additional measures are taken to ensure that the publish/subscribe infrastructure is to a degree independent of the underlying software stack. A truly holistic security solution could explicitly include such security considerations for the underlying infrastructure in the form of requirements, or perhaps even as required security patterns (cf. Fernandez and Uzunov, 2012) that would complement the ones referred to above in some future publish/subscribe security pattern language.

Appendix A

In Table 26 we attempt to briefly describe, in an indicative fashion only, the patterns appearing with a star (*) in the analy-

ses of solutions throughout Sections 4 and 5 that are as yet undocumented in the literature. We should point out that *Secure routing* in particular can be considered to be not just a compound pattern, but a whole pattern family, as is done in (Uzunov and Fernandez, 2016); however, including the details of this even indicatively is beyond the scope of the present paper. Descriptions of the remaining (documented) security patterns can be found in the following references: Fernandez (2013), Hafiz et al. (2011), Hafiz (2013), Schumacher et al. (2006), Steel et al. (2005), Uzunov and Fernandez (2016), Uzunov et al. (2012a, 2015c). Hafiz et al. (2011) and the survey of Uzunov et al. (2012a) in particular provide summaries of a broad selection of patterns, and can be referred to for a quick overview in the first instance. A comprehensive listing featuring corresponding per-pattern references can be found in the later survey of Bunke et al. (2012).

REFERENCES

- AbdAllah E, Hassanein HS, Zulkernine M. A survey of security attacks in Information-Centric Networking. *IEEE Communications Surveys & Tutorials* 2015;17(3):1441–54.
- Abie H. 2009. Adaptive security and trust management for autonomic message-oriented middleware, in: *Procs. 6th international conference on mobile adhoc and sensor systems (MASS)*. IEEE, Macau, pp. 810–817.
- Abie H, Dattani I, Novkovic M, Bigham J, Topham S, Savola R. 2008. GEMOM – significant and measurable progress beyond the state of the art, in: *Procs. 3rd international conference on systems and networks communications (ICSNC)*. IEEE, Sliema, pp. 191–196. doi:10.1109/ICSNC.2008.33.
- Abie H, Savola RM, Dattani I. 2009. Robust, secure, self-adaptive and resilient messaging middleware for business critical systems, in: *Procs. ComputationWorld 2009*. IEEE, Athens, pp. 153–160. doi:10.1109/ComputationWorld.2009.26.

- Abie H, Savola RM, Bigham J, Dattani I, Rotondi D, Da Bormida G. Self-healing and secure adaptive messaging middleware for business-critical systems. *Int J Adv Secur* 2010;3(1 & 2):34–51.
- Ahlgren B, Dannewitz C, Imbrenda C, Kutscher D, Ohlman B. A survey of information-centric networking. *IEEE Commun Magazine* 2012;50(7):26–36.
- Altaweel A. Providing basic security mechanisms in a publish/subscribe system [Masters thesis]. University of Stuttgart; 2009.
- Aniello L, Baldoni R, Ciccotelli C, Di Luna GA, Frontali F, Querzoni L. The overlay scan attack: inferring topologies of distributed pub/sub systems through broker saturation. *ACM Press*; 2014. p. 107–17.
- Arnold D, Lister I. Elvin federation protocol 1.0, Internet-Draft, <<http://www.elvin.org/specs/draft-elvin-federation-10-00.txt>>; 2007 [accessed 05.16].
- Arkins R. Persistent Elvin For Mobile Devices, [BInfTech Honours thesis]. Australia: The University of Queensland; 2000.
- Avgeriou P, Zdun U. 2005. Architectural patterns revisited – a pattern language, in: *Procs. 10th European conference on pattern languages of programming (EuroPLOP 2005)*. Kloster Isree, Germany, pp. 1–39.
- Bacon J, Eysers D, Moody K, Pesonen L. Securing publish/subscribe for multi-domain systems. In: *Alonso G, editor. Middleware 2005, lecture notes in computer science*. Berlin Heidelberg: Springer; 2005. p. 1–20.
- Bacon J, Eysers DM, Singh J, Pietzuch PR. 2008. Access control in publish/subscribe systems, in: *Proceedings of the second international conference on distributed event-based systems*. ACM, pp. 23–34.
- Bacon J, Eysers D, Singh J, Shand B, Migliavacca M, Pietzuch P. Security in multi-domain event-based systems. *Inf Technol (it – Methoden und innovative Anwendungen der Informatik und Informationstechnik)* 2009;51(5):277–84. doi:10.1524/itit.2009.0552.
- Bakken DE, Bose A, Hauser CH, Whitehead DE, Zweigle GC. Smart generation and transmission with coherent, real-time data. *P IEEE* 2011;99:928–51. doi:10.1109/JPROC.2011.2116110.
- Baldoni R, Beraldi R, Tucci Piergiovanni S, Virgillito A. On the modelling of publish/subscribe communication systems. *Concurr Comput Pract Exper* 2005;17(12):1471–95. doi:10.1002/cpe.879.
- Baldoni R, Querzoni L, Tarkoma S, Virgillito A. Distributed event routing in publish/subscribe systems. In: *Garbinato B, Miranda H, Rodrigues L, editors. Middleware for network eccentric and mobile applications*. Berlin Heidelberg: Springer; 2009. p. 219–44.
- Banavar G, Kaplan M, Shaw K, Strom RE, Sturman DC, Tao W. 1999. Information flow based event distribution middleware, in: *Procs. 19th IEEE international conference on distributed computing systems workshops on electronic commerce and web-based applications/middleware*, pp. 114–121. doi:10.1109/ECMDD.1999.776424.
- Barazzutti R, Felber P, Mercier H, Onica E, Rivière E. 2012. Thrifty privacy: efficient support for privacy-preserving publish/subscribe, in: *Procs. 6th int. conf. on distributed event-based systems (DEBS)*. ACM, pp. 225–236.
- Behnel S, Fiege L, Muhl G. 2006. On quality-of-service and publish-subscribe, in: *Procs. 26th IEEE international conference on distributed computing systems workshops (ICDCS Workshops 2006)*. IEEE, pp. 20–20.
- Bellavista P, Corradi A, Reale A. Quality of service in wide scale publish/subscribe systems. *IEEE Communications Surveys & Tutorials*. 2014;16(3):1591–1616. doi:10.1109/SURV.2014.031914.00192.
- Belokosztolszki A, Eysers DM, Pietzuch PR, Bacon J, Moody K. Role-based access control for publish/subscribe middleware architectures. In: *Procs. 2nd International Workshop on Distributed Event-Based Systems (DEBS)*. ACM; 2003. p. 1–8.
- Bernard S, Potop-Butucaru MG, Tixeuil S. A framework for secure and private P2P publish/subscribe. In: *Stabilization, safety, and security of distributed systems*. Springer; 2010. p. 531–45.
- Blanco R, Alencar P. Event models in distributed event based systems. In: *Hinze AM, Buchmann A, editors. Principles and applications of distributed event-based systems*. IGI Global; 2010. p. 19–42.
- Blanco R, Wang J, Alencar P. 2008. A metamodel for distributed event based systems, in: *Procs. 2nd international conference on distributed event-based systems (DEBS)*. ACM, pp. 221–232.
- Blanco RM. Process models for distributed event-based systems [Ph.D. thesis]. University of Waterloo; 2010.
- Bruni R, Corradini A, Gadducci F, Lafuente AL, Vandin A. A conceptual framework for adaptation. In: *de Lara J, Zisman A, editors. Fundamental approaches to software engineering (procs. fase 2012), lecture notes in computer science 7212*. Springer: Berlin Heidelberg; 2012. p. 240–54.
- Bu T, Norden S, Woo T. A survivable DoS-resistant overlay network. *Comput Networks* 2006;50(9):1281–301.
- Bunke M, Koschke R, Sohr K. Organizing security patterns related to security and pattern recognition requirements. *Int J Adv Secur* 2012;5(1 & 2):46–67.
- Burdusel L. 2011. Cryptographic protocol for secure dissemination of notification messages, in: *Procs. 2011 IEEE international conference on intelligent computer communication and processing (ICCP)*. IEEE, pp. 399–405.
- Buschmann F, Henney K, Schmidt DC. Pattern-oriented software architecture. In: *A pattern language for distributed computing*, vol. 4. 1st ed. Wiley; 2007.
- Carvalho HS, Murphy AL, Heinzelman WB, Coelho CJN Jr, 2004. Network-based distributed systems middleware, in: *Middleware Workshops*. pp. 13–20.
- Castro M, Druschel P, Kermarrec A-M, Rowstron AIT. Scribe: a large-scale and decentralized application-level multicast infrastructure. *IEEE J Select Areas Commun* 2002;20(8):1489–99.
- Chaabane A, Jmaiel M. 2009. A secure content-based publish/subscribe middleware. *IEEE*, pp. 619–622.
- Chakrabarti A. *Grid computing security*. 2007 ed. Berlin: Springer; 2007.
- Chakravarthy R, Hauser C, Bakken DE. Long-lived authentication protocols for process control systems. *Int J Critical Infras Protect* 2010;3(3–4):174–81. doi:10.1016/j.ijcip.2010.10.001.
- Chapin PC, Skalka C, Wang XS. Authorization in trust management: features and foundations. *ACM Comput Surv* 2008;40(3):9:1–48. doi:10.1145/1380584.1380587.
- Choi S, Ghinita G, Bertino E. A privacy-enhancing content-based publish/subscribe system using scalar product preserving transformations. In: *Database and expert systems applications*. Springer; 2010. p. 368–84.
- Corsaro A, Querzoni L, Scipioni S, Piergiovanni ST, Virgillito A. 2006. Quality of service in publish/subscribe middleware. *Global Data Management* 19, 20.
- Cristea V, Pop F, Dobre C, Costan A. Distributed architectures for event-based systems. In: *Helmer S, Poulouvasilis A, Xhafa F, editors. Reasoning in event-based distributed systems, studies in computational intelligence*. Berlin Heidelberg: Springer; 2011. p. 11–45.
- Crescenzo G, Coan B, Schultz J, Tsang S, Wright RN. Privacy-preserving publish/subscribe: efficient protocols in a distributed model. In: *Garcia-Alfaro J, et al., editors. Data Privacy Management and Autonomous Spontaneous Security*, Springer LNCS 8247. Springer-Verlag; 2014. p. 114–33.
- Daubert J, Fischer M, Grube T, Schiffner S, Kikiras P, Mühlhäuser M. AnonPubSub: Anonymous publish-subscribe overlays. *Computer Communications* 2016;76:42–53.

- Daubert J, Fischer M, Schiffner S, Mühlhäuser M. Distributed and anonymous publish-subscribe. In: Lopez J, Huang X, Sandhu R, editors. *Network and System Security*. Springer LNCS 7873. Springer Berlin-Heidelberg; 2013. p. 685–91.
- Dikanski A, Steinegger R, Abeck S. 2012. Identification and implementation of authentication and authorization patterns in the Spring security framework, in: *Procs. 6th international conference on emerging security information, systems and technologies (SECURWARE)*. IARIA, Rome, Italy, pp. 14–20.
- Dini G, Lo Duca A. 2009. On securing publish-subscribe systems with security groups, in: *IEEE symposium on computers and communications (ISCC 2009)*. IEEE, pp. 532–537.
- Dionysiou I. *Dynamic and composable trust for indirect interactions* [Ph.D. thesis]. Washington State University; 2006.
- Dionysiou I, Frincke D, Bakken D, Hauser C. An approach to trust management: challenges for critical infrastructures. In: Lopez J, Hämmmerli BM, editors. *Critical information infrastructures security (Procs. CRITIS 2007)*, LNCS 5141. Berlin Heidelberg: Springer; 2008. p. 173–84.
- Docking M, Uzunov AV, Fiddymont C, Brain R, Hewett S, Blucher L. 2015. UNISON: towards a middleware architecture for autonomous cyber defence, in: *Procs. 24th Australasian software engineering conference (ASWEC)*, IEEE computer society, Adelaide, Australia, pp. 203–212.
- Dube P, Halim N, Karenos K, Kim M, Liu Z, Parthasarathy S, et al. Harmony: holistic messaging middleware for event-driven systems. *IBM Syst J* 2008;47(2):281–7.
- Endler M, Baptista G, Silva LD, Vasconcelos R, Malcher M, Pantoja V, et al., 2011. ContextNet: context reasoning and sharing middleware for large-scale pervasive collaboration and social networking, in: *Procs. 12th international middleware conference workshop on posters and demos track (PDT)*, ACM, Lisbon, Portugal, pp. 2:1–2:2.
- Esposito C, Ciampi M. On security in publish/subscribe services: a survey. *IEEE Commun Surv Tutor* 2015;17(2):966–97. doi:10.1109/COMST.2014.2364616.
- Esposito C, Cotroneo D, Russo S. On reliability in publish/subscribe services. *Comput Networks* 2013;57(5):1318–43. doi:10.1016/j.comnet.2012.10.023.
- Eugster P. Type-based publish/subscribe: concepts and experiences. *ACM Trans Prog Lang Syst* 2007;29(1):6:1–50. doi:10.1145/1180475.1180481.
- Eugster PT, Felber PA, Guerraoui R, Kermarrec A-M. The many faces of publish/subscribe. *ACM Comput Surveys (CSUR)* 2003;35(2):114–31.
- Eyers DM, Vargas L, Singh J, Moody K, Bacon J. 2010. Relational database support for event-based middleware functionality, in: *Procs. 4th int. conference on distributed event-based systems (DEBS)*. ACM, pp. 160–171.
- Fernandez EB. *Security patterns in practice: designing secure architectures using software patterns*. 1st ed. Wiley; 2013.
- Fernandez EB, Uzunov AV. Secure middleware patterns. In: Xiang Y, Lopez J, Kuo C-CJ, Zhou W, editors. *Cyberspace safety and security, lecture notes in computer science 7672*. Springer: Berlin Heidelberg; 2012. p. 470–82.
- Fernandez EB, Monge R, Hashizume K. Building a security reference architecture for cloud systems. *Requir Eng* 2016;21(2):225–49. doi:10.1007/s00766-014-0218-7.
- Fidler E. *PADRES: a distributed content-based publish/subscribe system* [Ph.D. thesis]. University of Toronto; 2006.
- Fiege L, Zeidler A, Buchmann A, Kilian-Kehr R, Mühl G. 2004. Security aspects in publish/subscribe systems, in: *Procs. 3rd international workshop on distributed event-based systems (DEBS)*. IET, Edinburgh, UK, pp. 44–49.
- Fischer T. *Quality-of-Service-Aware Configuration of Distributed Publish-Subscribe Systems*, [PhD Thesis]. Germany: Friedrich-Alexander Universität Erlangen-Nürnberg; 2014.
- Fotiou N, Marias GF, Polyzos GC. Towards a secure rendezvous network for future publish/subscribe architectures. In: *Future internet-FIS 2010*. Springer; 2010a. p. 49–56 LNCS 6369.
- Fotiou N, Marias GF, Polyzos GC. 2010b. Fighting spam in publish/subscribe networks using information ranking, in: *Procs. 6th EURO-NF conference on next generation internet (NGI)*. IEEE, pp. 1–6.
- García J, Jaeger MA, Mühl G, Borrell J. Decoupling components of an attack prevention system using publish/subscribe. In: *Intelligence in communication systems*. Springer; 2005. p. 87–97.
- Gjermundrød H, Gjermundrød H, Bakken DE, Hauser CH, Bose A. GridStat: a flexible qos-managed data dissemination framework for the power grid. *IEEE Trans Power Delivery* 2009;24(1):136–43.
- Gonçalves JF, da Silva e Silva FJ, Vasconcelos RO, Baptista GLB, Endler M. 2013. A security infrastructure for massive mobile data distribution. *Procs. 11th ACM international symposium on mobility management and wireless access (MobiWac)*, ACM Press, pp. 41–50. doi:10.1145/2508222.2508237.
- Hafiz M. A pattern language for developing privacy enhancing technologies. *Softw Pract Exp* 2013;43(7):769–87.
- Hafiz M, Adamczyk P, Johnson R. 2011. Growing a pattern language (for security). In: *Procs. 18th conference on pattern languages of programs (PLOP)*.
- Halkidis ST, Tsantalis N, Chatzigeorgiou A, Stephanides G. Architectural risk analysis of software systems based on security patterns. *IEEE Trans Depend Secure Comput* 2008;5(3):129–42.
- Hall S, McQuay W. 2011. Fundamental features of a unified trust model for distributed systems, in: *Procs. 2011 IEEE national aerospace and electronics conference (NAECON)*. IEEE, Dayton, OH, pp. 139–145.
- Hauser CH, Bakken DE, Dionysiou I, Gjermundrød KH, Irava V, Helkey J, et al. Security, trust, and QoS in next-generation control and communication for large power systems. *Int J Critical Infras* 2008;4(1):3–16.
- Hein P, Biswas D, Martucci LA, Muhlhauser M. 2011. Conflict detection and lifecycle management for access control in publish/subscribe systems, in: *Procs. 13th international symposium on high-assurance systems engineering (HASE)*. IEEE, pp. 104–111.
- Heyman T, Scandariato R, Huygens C, Joosen W. 2008. Using security patterns to combine security metrics, in: *Procs. 3rd international conference on availability, reliability and security (ARES)*. IEEE, Barcelona, Spain, pp. 1156–1163.
- Huang Y, Garcia-Molina H. Publish/subscribe in a mobile environment. *Wireless Netw* 2004;10(6):643–52.
- Hummer W, Inzinger C, Leitner P, Satzger B, Dustdar S. 2012. Deriving a unified fault taxonomy for event-based systems, in: *Proceedings of the 6th ACM international conference on distributed event-based systems (DEBS)*. ACM, pp. 167–78.
- IBM. The Gryphon project, <<http://www.research.ibm.com/distributedmessaging/gryphon.html>>; 2014 [accessed 2016].
- Ion M, Russello G, Crispo B. Design and implementation of a confidentiality and access control solution for publish/subscribe systems. *Computer Networks* 2012;56:2014–37.
- Jacobsen H-A, Cheung AKY, Li G, Maniymaran B, Muthusamy V, Kazemzadeh RS. 2010. The PADRES publish/subscribe system. *Jaeger MA. Self-Managing Publish/Subscribe Systems*, [PhD Thesis]. Germany: TU Berlin; 2007.
- Jaeger MA, Parzyjegl H, Mühl G, Herrmann K. 2007. Self-organizing broker topologies for publish/subscribe systems, in: *Proceedings of the 2007 ACM symposium on applied computing (SAC)*. ACM, pp. 543–550.
- Judge P, Ammar M. Security issues and solutions in multicast content distribution: A survey. *IEEE Netw* 2003;17:30–6.

- Kanstrén T, Savola R. 2010. Definition of core requirements and a reference architecture for a dependable, secure and adaptive distributed monitoring framework, in: *Procs. 3rd international conference on dependability (DEPEND)*. IEEE, Venice, pp. 154–163. doi:10.1109/DEPEND.2010.31.
- Kanstrén T, Savola R, Haddad S, Hecker A. An adaptive and dependable distributed monitoring framework. *Int J Adv Secur* 2011;4(1 & 2):80–94.
- Karenos K, Kim M, Lei H, Yang H, Ye F. 2009. Providing quality of service in wide-area publish/subscribe systems: poster, in: *Procs. 10th ACM/IFIP/USENIX int. conference on middleware*. Springer-Verlag New York, p. 31.
- Khurana H, Koleva RK. Scalable security and accounting services for content-based publish/subscribe Systems. *Int J E-Business Res* 2006;2(3):1–16. doi:10.4018/jebr.2006070101.
- Lagutin D, Visala K, Zahemszky A, Burbridge T, Marias GF. 2010. Roles and security in a publish/subscribe network architecture, in: *Computers and Communications (ISCC)*, 2010 IEEE Symposium on. IEEE, pp. 68–74.
- Lee AJ, Winslett M, Perano KJ. TrustBuilder2: a reconfigurable framework for trust negotiation. In: Ferrari E, Li N, Bertino E, Karabulut Y, editors. *Trust management III, IFIP advances in information and communication technology 300*. Berlin Heidelberg: Springer; 2009. p. 176–95.
- Li G, Muthusamy V, Jacobsen H-A. A distributed service-oriented architecture for business process execution. *ACM Trans Web* 2010;4(1):2:1–33. doi:10.1145/1658373.1658375.
- Liu Y, Plale B. 2003. Survey of publish subscribe event systems. Computer Science Dept, Indian University 16.
- Lu X, Yang T, Liao Z, Li X, Wang Y, Liu W, et al., 2008. A Novel QoS-Enable Real-Time Publish-Subscribe Service. *IEEE*, pp. 19–26. doi:10.1109/ISPA.2008.61.
- Mahambre SP, Kumar M, Bellur U. A taxonomy of QoS-aware, adaptive event-dissemination middleware. *IEEE Internet Comput* 2007;11(4):35–44.
- Maji AK, Bagchi S. v-CAPS: a confidentiality and anonymity preserving routing protocol for content-based publish-subscribe networks. In: *Security and privacy in communication networks*. Springer; 2012. p. 281–302.
- Marsh MA, Schneider FB. CODEX: a robust and secure secret distribution system. *IEEE Trans Depend Secure Comput* 2004;1(1):34–47. doi:10.1109/TDSC.2004.3.
- Martins J, Duarte S. Routing algorithms for content-based publish/subscribe systems. *IEEE Commun Surv Tutor* 2010;12(1):39–58. doi:10.1109/SURV.2010.020110.00065.
- Mayer TR, Brunie L, Coquil D, Kosch H. On reliability in publish/subscribe systems: a survey. *Int J Parall Emerg Distrib Syst* 2012;27(5):369–86. doi:10.1080/17445760.2012.697162.
- Meier R. Event-based middleware for collaborative ad hoc applications [Ph.D. thesis]. Department of Computer Science, Trinity College; 2003.
- Meier R, Cahill V. Taxonomy of distributed event-based programming systems. *Comput J* 2005;48(5):602–26. doi:10.1093/comjnl/bxh120.
- Migliavacca M, Papagiannis I, Eysers DM, Shand B, Bacon J, Pietzuch P. DEFCON: high-performance event processing with information security. In: *Procs. 2010 USENIX Annual Technical Conference*. USENIX Association; 2010a.
- Migliavacca M, Papagiannis I, Eysers DM, Shand B, Bacon J, Pietzuch P. Distributed middleware enforcement of event flow security policy. In: *Proceedings of the 11th ACM/IFIP/USENIX International Conference on Middleware*. Springer LNCS 6452. Springer-Verlag; 2010b. p. 334–54.
- Minami K, Lee AJ, Winslett M, Borisov N. 2008. Secure aggregation in a publish-subscribe system, in: *Proceedings of the 7th ACM workshop on privacy in the electronic society*. ACM, pp. 95–104.
- Minami K, Borisov N, Gunter CA. 2009. Safety in discretionary access control for logic-based publish-subscribe systems, in: *Procs. 14th ACM symposium on access control models and technologies*. ACM, pp. 3–12.
- Möckel C, Abdallah A. 2010. Threat modeling approaches and tools for securing architectural designs of an e-banking application, in: *Procs. 6th international conference on information assurance and security (IAS)*. Presented at the 2010 sixth international conference on information assurance and security (IAS), pp. 149–154.
- Mühl G, Fiege L, Pietzuch P. Distributed event-based systems. 1st ed. Berlin: Springer; 2006.
- Myagmar S, Lee A, Yurcik W. 2005. Threat modeling as a basis for security requirements. Presented at the Procs. 2005 ACM workshop on storage security and survivability (StorageSS), ACM Press, pp. 94–102.
- Nabeel M, Appel S, Bertino E, Buchmann A. 2013. Privacy preserving context aware publish subscribe systems, in: *Procs. 7th international conference (NSS)*. Springer, Madrid, Spain, pp. 465–478.
- Nabeel M. Privacy Preserving Access Control on Third-Party Data Management Systems, [PhD Thesis and CERIAS Tech Report 2012-12]. USA: Purdue University; 2012.
- Nguyen T-D, Huh E-N. 2011. An efficient key management for secure multicast in sensor-cloud. *IEEE*, pp. 3–9.
- OASIS. Web services notification (WSN), <<https://www.oasis-open.org/committees/wsn/>>; 2006 [accessed 12.15].
- OMG. DDS security specification, <<http://www.omg.org/spec/DDS-SECURITY/>>; 2014a [accessed 07.15].
- OMG. Data distribution service (DDS), <<http://www.omg.org/spec/DDS/>>; 2014b [accessed 07.15].
- OMG. The real-time publish-subscribe wire protocol, <<http://www.omg.org/spec/DDS-RTSPS/>>; 2014c [accessed 07.15].
- Opyrchal L, Prakash A. Secure Distribution of Events in Content-Based Publish Subscribe Systems. In: *Procs. 10th USENIX Security Symposium*. Washington DC, USA; 2001. p. 281–96.
- Opyrchal L, Prakash A. Publish subscribe middleware. In: Prabhuv, Kumara S, Kamath M, editors. *Scalable enterprise systems: an introduction to recent advances, integrated series in information systems*. US: Springer; 2003. p. 249–85.
- Opyrchal L, Prakash A, Agrawal A. Supporting privacy policies in a publish-subscribe substrate for pervasive environments. *J Netw* 2007;2(1):17–26.
- Paganelli F, Vannuccini G, Parlanti D, Giuli D, Cianchi P. 2011. GEMOM middleware self-healing and fault-tolerance: a highway tolling case study, in: *Procs. 6th international conference on systems and networks communications (ICSNC)*. pp. 136–142.
- Pal P, Lauer G, Khoury J, Hoff N, Loyall J. P3S: a privacy preserving publish-subscribe middleware. In: *Middleware 2012*. Springer; 2012. p. 476–95.
- Pallickara S, Fox G. NaradaBrokering: a distributed middleware framework and architecture for enabling durable peer-to-peer grids. In: Endler M, Schmidt D, editors. *Middleware 2003, lecture notes in computer science*. Berlin Heidelberg: Springer; 2003. p. 41–61.
- Pallickara S, Pierce M, Fox G, Yan Y, Huang Y. 2005. A security framework for distributed brokering systems. *Community Grids Laboratory Technical Report*.
- Pallickara S, Pierce M, Gadgil H, Fox G, Yan Y, Huang Y. 2006. A framework for secure end-to-end delivery of messages in publish/subscribe systems, in: *Proceedings of the 7th IEEE/ACM international conference on grid computing*. IEEE Computer Society, pp. 215–222.
- Pallickara S, Ekanayake J, Fox G. 2007. A scalable approach for the secure and authorized tracking of the availability of entities in distributed systems, in: *Procs. 2007 IEEE international*

- parallel and distributed processing symposium (IPDPS). IEEE, pp. 1–10.
- Pallickara S, Fox G, Gadgil H. On the secure creation, organisation and discovery of topics in distributed publish/subscribe systems. *Int J High Perform Comput Netw* 2008;5(3): 156–67.
- Parzyjeglą H. Engineering publish/subscribe systems and event-driven applications [Ph.D. thesis]. University of Rostock; 2014.
- Parzyjeglą H, Jaeger MA, Muhl G, Weis T. 2008. Model-driven development and adaptation of autonomous control applications. *Distributed Systems Online*, IEEE 9, 2–2.
- Parzyjeglą H, Graff D, Schröter A, Richling J, Mühl G. Design and implementation of the Rebeca publish/subscribe middleware. In: *From active data management to event-based systems and more*. Springer; 2010. p. 124–40.
- Paschke A, Vincent P, Alves A, Moxey C. 2012. Tutorial on advanced design patterns in event processing, in: *Proceedings of the 6th ACM international conference on distributed event-based systems*. ACM, pp. 324–334.
- Pesonen LI. 2008. A capability-based access control architecture for multi-domain publish/subscribe systems. University of Cambridge, Computer Laboratory, Technical Report.
- Pesonen LI, Bacon J. 2005. Secure event types in content-based, multi-domain publish/subscribe systems, in: *Proceedings of the 5th international workshop on software engineering and middleware*. ACM, pp. 98–105.
- Pesonen LI, Eysers DM, Bacon J. 2006. A capability-based access control architecture for multi-domain publish/subscribe systems, in: *Applications and the Internet, 2006. SAINT 2006. International Symposium on*. IEEE, p. 7–pp.
- Pesonen LI, Eysers DM, Bacon J. Access control in decentralised publish/subscribe systems. *J Netw* 2007;2(2):57–67.
- Pietzuch P, Eysers D, Kounev S, Shand B. 2007. Towards a common API for publish/subscribe, in: *Proceedings of the 2007 inaugural international conference on distributed event-based systems*. ACM, pp. 152–157.
- Pietzuch PR. 2002. Event-based middleware: a new paradigm for wide-area distributed systems, in: *6th CaberNet Radicals Workshop*.
- Pietzuch PR. 2004. Hermes: a scalable event-based middleware. University of Cambridge PhD thesis and TR590.
- Rahaman MA, Roudier Y, Schaad A, Oualha N, Önen M. A publish/subscribe model for secure content driven XML dissemination. *J Inf Assur Secur* 2010;6(1): Available online: <<http://www.eurecom.fr/publication/3089>>.
- Rao W, Chen L, Yuan M, Tarkoma S, Mei H. Subscription privacy protection in topic-based pub/sub. In: *Database systems for advanced applications*. Springer; 2013. p. 361–76.
- Riehle D. Lessons Learned from Using Design Patterns in Industry Projects. *Trans Pattern Lang Program* 2011;2: 1–15.
- Russello G, Mostarda L, Dulay N. A policy-based publish/subscribe middleware for sense-and-react applications. *J Syst Softw* 2011;84:638–54. doi:10.1016/j.jss.2010.10.023.
- Salehie M, Tahvildari L. Self-adaptive software: landscape and research challenges. *ACM Trans Auto Adapt Syst* 2009;4(2):14:1–42. doi:10.1145/1516533.1516538.
- Sandhu R. The authorization leap from rights to attributes: maturation or chaos? (Keynote). In: *Proceedings of the 17th ACM Symposium on Access Control Models and Technologies (SACMAT)*. ACM; 2012. p. 69–70.
- Savinov S, Alencar P. 2014. A risk management approach for distributed event-based systems. Technical Report CS-2014-06, University of Waterloo.
- Savola RM, Abie H. 2009a. Identification of basic measurable security components for a distributed messaging system, in: *Procs. 3rd international conference on emerging security information, systems and technologies (SECURWARE)*. IEEE, Athens, Glyfada, pp. 121–128. doi:10.1109/SECURWARE.2009.26.
- Savola RM, Abie H. Development of measurable security for a distributed messaging system. *Int J Adv Secur* 2009b;2(4):358–80.
- Savola RM, Heinonen P. Increasing measurability and meaningfulness of adaptive security monitoring by system architectural design and mechanisms. *Int J Adv Syst Measur* 2011;4(1 & 2):1–19.
- Savola RM, Abie H, Bigham J, Rotondi D. 2010. Innovations and advances in adaptive secure message oriented middleware, in: *Procs. 30th international conference on distributed computing systems workshops (ICDCSW)*. IEEE, Genova, pp. 288–289. doi:10.1109/ICDCSW.2010.70.
- Schumacher M, Fernandez EB, Hybertson D, Buschmann F, Sommerlad P. *Security Patterns: Integrating Security and Systems Engineering*. 1st ed. Wiley; 2006.
- Schmidt D, Stal M, Rohnert H, Buschmann F. Pattern-oriented software architecture. In: *Patterns for concurrent and networked objects*, vol. 2. 1st ed. Wiley; 2000.
- Segall B, Arnold D, Boot T, Henderson M, Phelps T. 2000. Content based routing with Elvin4, in: *Procs. of AUUG2k*.
- Shand B, Pietzuch P, Papagiannis I, Moody K, Migliavacca M, Eysers DM, et al. Security policy and information sharing in distributed event-based systems. In: *Reasoning in event-based distributed systems*. Springer; 2011. p. 151–72.
- Shikfa A, Önen M, Molva R. Privacy-preserving content-based publish/subscribe networks. In: *Gritzalis D, Lopez J, editors. Emerging Challenges for Security, Privacy and Trust (Procs. SEC 2009)*. Springer; 2009. p. 270–82.
- Singh J, Bacon J. Event-based data dissemination control in healthcare. In: *Weerasinghe D, editor. Electronic healthcare, lecture notes of the institute for computer sciences, social informatics and telecommunications engineering*. Berlin Heidelberg: Springer; 2009. p. 167–74.
- Singh J, Vargas L, Bacon J, Moody K. 2008. Policy-based information sharing in publish/subscribe middleware. IEEE, pp. 137–144. doi:10.1109/POLICY.2008.17.
- Singh J, Eysers DM, Bacon J. 2011. Disclosure control in multi-domain publish/subscribe systems, in: *Proceedings of the 5th ACM international conference on distributed event-based system*. ACM, pp. 159–170.
- Singh J, Bacon J, Eysers D. Policy enforcement within emerging distributed, event-based systems. *ACM Press*; 2014. p. 246–55. doi:10.1145/2611286.2611310.
- Sloman M. Policy driven management for distributed systems. *J Netw Syst Manage* 1994;2(4):333–60.
- Smith B, Williams L. 2012. On the effective use of security test patterns. In: *Procs. 6th IEEE international conference on software security and reliability (SERE)*, Gaithersburg, MD, USA, IEEE, pp. 108–117.
- Solum E. Achieving over-the-wire configurable confidentiality, integrity, authentication and availability in GridStat's status dissemination [Masters thesis]. Washington State University; 2007.
- Solum E, Hauser C, Chakravarthy R, Bakken D. 2009. Modular over-the-wire configurable security for long-lived critical infrastructure monitoring systems, in: *Proceedings of the third ACM international conference on distributed event-based systems*. ACM, p. 13.
- Srivatsa M, Liu L. 2005. Securing publish-subscribe overlay services with EventGuard, in: *Procs. 12th ACM conference on computer and communications security (CCS)*. ACM, pp. 289–298. doi:10.1145/1102120.1102158.
- Srivatsa M, Liu L. 2007. Secure event dissemination in publish-subscribe networks, in: *Procs. 27th international conference on distributed computing systems (ICDCS)*. pp. 22–22. doi:10.1109/ICDCS.2007.136.

- Srivatsa M, Liu L, Iyengar A. EventGuard: a system architecture for securing publish-subscribe networks. *ACM Trans Comput Syst* 2011;29(4):10:1–40. doi:10.1145/2063509.2063510.
- Steel C, Nagappan R, Lai R. Core security patterns: best practices and strategies for J2EE(TM), web services, and identity management. Prentice Hall; 2005.
- Sutton P, Arkins R, Segall B. 2001. Supporting disconnectedness-transparent information delivery for mobile and invisible computing, in: Proceedings of the 1st IEEE/ACM international symposium on cluster computing and the grid, pp. 277–285.
- Tajalli H, Garcia J, Edwards G, Medvidovic N. 2010. PLASMA: a plan-based layered architecture for software model-driven adaptation, in: Procs. 2010 IEEE/ACM international conference on automated software engineering (ASE). ACM, pp. 467–476.
- Tariq MA. Non-functional requirements in publish/subscribe systems [Ph.D. thesis]. University of Stuttgart; 2013.
- Tariq MA, Koldehofe B, Altaweel A, Rothermel K. 2010. Providing basic security mechanisms in broker-less publish/subscribe systems, in: Procs. 4th ACM international conference on distributed event-based systems (DEBS). ACM, pp. 38–49. doi:10.1145/1827418.1827425.
- Tariq MA, Koldehofe B, Rothermel K. Securing broker-less publish/subscribe systems using identity-based encryption. *IEEE Trans Parallel Distrib Syst* 2014;25(2):518–28. doi:10.1109/TPDS.2013.256.
- Tarkoma S. Mobile middleware: supporting applications and services. Chichester, U.K.: Wiley; 2009.
- Tarkoma S. Publish / subscribe systems: design and principles. London; Hoboken, N.J.: Wiley; 2013.
- Tarkoma S, Raatikainen K. 2006. State of the art review of distributed event systems. Minema Events Report C0-04.
- Uzunov AV, Fernandez EB. An extensible pattern-based library and taxonomy of security threats for distributed systems. *Comput Stand Interf* 2014;36(4):734–47. doi:10.1016/j.csi.2013.12.008. Special Issue on Security in Information Systems: Advances and New Challenges.
- Uzunov AV, Fernandez EB. 2016. Cryptography-based security patterns and security solution frames for networked and distributed systems. Submitted for publication.
- Uzunov AV, Fernandez EB, Falkner K. Securing distributed systems using patterns: a survey. *Comput Secur* 2012a;31(5):681–703. doi:10.1016/j.cose.2012.04.005.
- Uzunov AV, Fernandez EB, Falkner K. Engineering security into distributed systems: a survey of methodologies. *J Univ Comput Sci* 2012b;18(20):2920–3006. doi:10.3217/jucs-018-20-2920t.
- Uzunov AV, Falkner K, Fernandez EB. 2013. Decomposing distributed software architectures for the determination and incorporation of security and other non-functional requirements. In: Procs. of the 22nd Australian software engineering conference (ASWEC). Melbourne, Australia: IEEE Computer Society, pp. 30–39.
- Uzunov AV, Fernandez EB, Falkner K. ASE: a comprehensive pattern-driven security methodology for distributed systems. *Comput Stand Interf* 2015a;41:112–37. doi:10.1016/j.csi.2015.02.011.
- Uzunov AV, Falkner K, Fernandez EB. A comprehensive pattern-oriented approach to engineering security methodologies. *Inf Softw Technol* 2015b;57:217–47. doi:10.1016/j.infsof.2014.09.001.
- Uzunov AV, Fernandez EB, Falkner K. Security solution frames and security patterns for authorization in distributed, collaborative systems. *Comput Secur* 2015c;55:193–234. doi:10.1016/j.cose.2015.08.003.
- Virgillito A. Publish/Subscribe Communication Systems: from Models to Applications, [PhD Thesis]. Italy: Department of Computer and Systems Engineering, University of Rome “La Sapienza”; 2003.
- Visala K, Lagutin D, Tarkoma S. 2009. LANES: an inter-domain data-oriented routing architecture, in: Proceedings of the 2009 workshop on re-architecting the internet. ACM, pp. 55–60.
- Visala K, Lagutin D, Tarkoma S. Security design for an inter-domain publish/subscribe architecture. Springer; 2011.
- Voelter M, Kircher M, Zdun U. Remoting Patterns: Foundations of Enterprise, Internet and Realtime Distributed Object Middleware. Wiley; 2005.
- Wang J, Bigham J. Anomaly detection in the case of message oriented middleware. In: Procs. 2008 Workshop on Middleware Security (MidSec). ACM; 2008. p. 40–2.
- Wang C, Carzaniga A, Evans D, Wolf AL. 2002. Security issues and requirements for internet-scale publish-subscribe systems, in: System Sciences, 2002. HICSS. Proceedings of the 35th annual Hawaii international conference on. IEEE, pp. 3940–3947.
- Wang C, Liu Y, Gao J, Ma X. An encryption mechanism based on redundant attributes in multi-domain publish/subscribe networks. *J Comput Inf Syst* 2013;9(7):2861–71.
- Wang J, Bigham J, Murciano B. 2010a. Towards a resilient message oriented middleware for mission critical applications, in: Procs. 2nd international conference on adaptive and self-adaptive systems and applications (ADAPTIVE). pp. 46–51.
- Wang J, Jiang P, Bigham J, Chew B, Novkovic M, Dattani I. 2010b. Adding resilience to message oriented middleware, in: Procs. 2nd international workshop on software engineering for resilient systems (SERENE). ACM, pp. 89–94.
- Weyns D, Malek S, Andersson J. FORMS: unifying reference model for formal specification of distributed self-adaptive systems. *ACM Trans Auto Adapt Syst* 2012;7(1):8:1–61. doi:10.1145/2168260.2168268.
- Weyns D, Schmerl B, Grassi V, Malek S, Mirandola R, Prehofer C, et al. On patterns for decentralized control in self-adaptive systems. In: de Lemos R, Giese H, Müller HA, Shaw M, editors. Software engineering for self-adaptive systems II, Lecture notes in computer science, vol. 7475. Berlin Heidelberg: Springer; 2013. p. 76–107.
- Wun A, Jacobsen H-A. A policy management framework for content-based publish/subscribe middleware. In: *Middleware* 2007. Springer; 2007. p. 368–88.
- Wun A, Cheung A, Jacobsen H-A. 2007. A taxonomy for denial of service attacks in content-based publish/subscribe systems, in: Proceedings of the 2007 inaugural international conference on distributed event-based systems (DEBS), ACM, pp. 116–127. doi:10.1145/1266894.1266917.
- Xiao Y. Security in distributed, grid, mobile, and pervasive computing. Boca Raton: Auerbach Publications; 2007.
- Yan Y, Huang Y, Fox G, Pallickara S, Pierce ME, Kaplan A, et al. Implementing a prototype of the security framework for distributed brokering systems. In: *Security and Management*. Citeseer; 2003. p. 212–18.
- Yang H, Kim M, Karenos K, Ye F, Lei H. Message-oriented middleware with QoS awareness. In: *Service-oriented computing*. Springer; 2009. p. 331–45.
- Yoon Y. Adaptation techniques for publish/subscribe overlays [Ph.D. thesis]. University of Toronto; 2013.
- Yuan E, Esfahani N, Malek S. A systematic survey of self-protecting software systems. *ACM Trans Auto Adapt Syst* 2014;8(4):17:1–41. doi:10.1145/2555611.
- Zhao Y, Sturman DC. Dynamic access control in a content-based publish/subscribe system with delivery guarantees. In: Procs. 26th IEEE International Conference on Distributed Computing Systems (ICDCS). IEEE Computer Society; 2006. p. 60–8.
- Zou X, Ramamurthy B, Magliveras SS. *Secure Group Communications Over Data Networks*. 1st ed. Springer; 2010.

Dr. Anton V. Uzunov works as a researcher for the Australian Defence Science and Technology Group (DST Group). He holds a

PhD in Computer Science, an Honours degree in Pure Mathematics and a Bachelor degree in Maths and Computer Science, all from the University of Adelaide. His research interests span the fields of software architecture, computer security, operating systems and parallel and distributed computing. Specific interests include the

application of software engineering principles for the improvement of software security, software analysis and design, and the construction of secure, reliable and adaptive architectures for various types of distributed systems.